

MANUAL TÉCNICO

Sistema de Gestión de Lavado Vehicular (Vehicle Washing)

Arquitectura de microservicios, modelo de datos, contratos de integración, seguridad y lógica de negocio

Tipo de manual	Manual técnico (desarrolladores y arquitectos de software)
Integrantes	Brayan Estiven Patiño Cabrera (líder de proyecto) Juan Camilo Guilombo Osorio (líder de frontend) Merari Urbano (líder de backend)
Ficha	3145556
Programa	Tecnólogo en Análisis y Desarrollo de Software (ADSO)
Instructor/a	Karol Daniela Correa
Centro de formación	Industria
Fecha	05 de octubre de 2026
Versión	2.0

Control de versiones

La Tabla 1 registra el historial de cambios del presente documento. Cada modificación debe agregar una fila con la versión, la fecha, la descripción del cambio y el responsable. No debe confundirse con el control de versiones del código fuente, que se describe en la sección 15.

Tabla 1

Historial de versiones del manual técnico

Versión	Fecha	Descripción del cambio	Responsable
1.0	04/10/2026	Creación inicial del manual técnico a partir de la documentación del proyecto (versión de la documentación del 02/10/2026).	Equipo desarrollador
2.0	05/10/2026	Se completan los datos de portada (ficha, instructora y centro de formación), se ajusta la denominación del sistema a Vehicle Washing y se corrige la descripción de la estrategia de migraciones (changelog maestro único).	Equipo desarrollador

Nota. Elaboración propia del equipo desarrollador.

Tabla de contenido

Control de versiones	2
4. Introducción	6
4.1 Fuentes y convenciones de este documento	6
5. Objetivo	7
5.1 Objetivo general	7
5.2 Objetivos específicos	7
6. Alcance técnico	8
6.1 Qué cubre este manual	8
6.2 Qué no cubre este manual	8
7. Descripción general del sistema	9
7.1 Resumen de la solución	9
7.2 Tipo de aplicación y componentes principales	9
7.3 Restricciones arquitectónicas	10
8. Tecnologías utilizadas	11
9. Arquitectura del sistema	13
9.1 Estilo arquitectónico	13
9.2 Puerta de enlace (API Gateway)	14
9.3 Arquitectura interna de cada servicio: hexagonal	14
9.4 Comunicación entre componentes	15
9.4.1 Comunicación síncrona con REST	15
9.4.2 Comunicación asíncrona con RabbitMQ	16
9.5 Preocupaciones transversales	16
9.5.1 Auditoría y ciclo de vida de los datos	16
9.5.2 Concurrencia	17
9.5.3 Transacciones	17
9.5.4 Registro, correlación y manejo de errores	17
9.5.5 Tiempo e idioma	18
9.5.6 Patrones no adoptados	18
10. Estructura del proyecto	19
10.1 Repositorio de documentación	19

10.2 Estructura interna prescrita para cada servicio	20
10.3 Artefactos de código mencionados por la documentación	20
11. Base de datos	22
11.1 Diagrama entidad-relación	22
11.2 Tablas principales	23
11.2.1 Propiedad de esquemas y reglas de llaves foráneas	23
11.2.2 Convenciones de modelado	24
11.2.3 Descripción de las tablas por esquema	25
11.3 Diccionario de datos	30
11.3.1 Esquema security	31
11.3.2 Esquema audit	33
11.3.3 Esquema customer	33
11.3.4 Esquema catalog	34
11.3.5 Esquema booking	35
11.3.6 Esquema execution	38
11.3.7 Esquema promotion	40
11.3.8 Esquema payment	42
11.3.9 Esquema notification	44
11.4 Reglas de integridad que la base de datos no puede imponer	45
11.5 Migraciones y datos iniciales	46
12. Módulos técnicos	48
12.1 security-service	48
12.2 customer-service	48
12.3 booking-service	48
12.4 operations-service	49
12.5 payment-service	49
12.6 notification-service	50
12.7 Ciclos de vida	50
13. API o rutas del sistema	52
13.1 Convenciones generales	52
13.2 Autenticación en la API	52
13.3 Endpoints documentados	53

13.3.1 Creación de una reserva	54
13.4 Eventos de dominio (RabbitMQ)	54
13.4.1 Contrato de notificaciones	56
13.5 Verificación y salud	56
14. Seguridad	57
14.1 Autenticación y control de sesiones	57
14.2 Autorización	57
14.3 Validación de entrada	58
14.4 Cifrado, secretos y comunicación	58
14.5 Lista de control OWASP Top 10	59
14.6 Protección de datos personales	59
14.7 Auditoría y eventos de seguridad	59
14.8 Riesgos aceptados	60
15. Control de versiones	61
15.1 Código fuente	61
15.2 Esquema de base de datos	61
15.3 Decisiones de arquitectura	61
16. Recomendaciones de mantenimiento	63
16.1 Entorno local y puesta en marcha	63
16.2 Variables de entorno y configuración	63
16.3 Actualizaciones y dependencias	63
16.4 Base de datos, copias de seguridad y recuperación	64
16.5 Pruebas	64
16.6 Diagnóstico de problemas frecuentes	64
16.7 Evolución recomendada	65
16.8 Decisiones pendientes	65
17. Glosario	66
18. Anexos	69
Anexo A. Lista de figuras pendientes de captura	69
Anexo B. Discrepancias detectadas en la documentación	69
Anexo C. Lista de chequeo de entrega	71
Referencias	72

4. Introducción

El Sistema de Gestión de Lavado Vehicular (Vehicle Washing) es una plataforma que permite reservar turnos de lavado, asignar operarios, ejecutar y calificar los servicios y verificar pagos realizados por código QR en un único establecimiento. Desde el punto de vista técnico, se construyó como un conjunto de seis microservicios de dominio escritos en Java 21 con Spring Boot, detrás de una puerta de enlace (API Gateway), con una base de datos SQL Server compartida físicamente y separada lógicamente por esquemas, y con mensajería asíncrona mediante RabbitMQ.

Este manual documenta cómo está construido el sistema: la arquitectura y las decisiones que la sustentan, el modelo de datos, la comunicación entre servicios, los contratos de API conocidos, la seguridad y las reglas de negocio que el código debe respetar. Está dirigido a desarrolladores, arquitectos, personal de soporte técnico e instructores que deban mantener o evolucionar el sistema.

La arquitectura se apoya en conceptos ampliamente documentados: la arquitectura de microservicios (Newman, 2021), el diseño guiado por el dominio (Evans, 2004), la arquitectura hexagonal de puertos y adaptadores (Cockburn, 2005) y el patrón de mensajería de publicación y suscripción (Hohpe y Woolf, 2003). Cuando estos conceptos aparecen en el texto se citan con el formato de las Normas APA, séptima edición, y se listan en la sección de referencias.

4.1 Fuentes y convenciones de este documento

El contenido se elaboró a partir del repositorio de documentación del proyecto (requisitos, historias de usuario, modelo de dominio, decisiones de arquitectura ADR-001 a ADR-012, modelo de datos, políticas de seguridad y guías de entorno). **El material analizado contiene documentación, no el código fuente de los servicios.** Por esa razón, los nombres de clases, paquetes y archivos que se citan son únicamente los que la documentación menciona de forma explícita, y la estructura interna de cada servicio se presenta como la estructura prescrita por la arquitectura (sección 10), no como un listado verificado del código. Antes de la entrega final, el equipo debe contrastar cada sección con el código de la versión entregada, como exige la guía ADSO (SENA, 2026).

Las fuentes de la documentación no siempre coinciden entre sí, porque el proyecto evolucionó durante su elaboración. Cuando se detectó una contradicción, este manual adopta la decisión más reciente y aceptada, e indica la discrepancia en una nota. El Anexo B reúne todas las discrepancias encontradas para que el equipo las resuelva.

5. Objetivo

5.1 Objetivo general

Documentar la arquitectura, las tecnologías, el modelo de datos, la comunicación entre servicios, la seguridad y las reglas de negocio del Sistema de Gestión de Lavado Vehicular, de modo que otro desarrollador pueda comprender, mantener y continuar el proyecto.

5.2 Objetivos específicos

- Describir el estilo arquitectónico (microservicios con propiedad lógica de datos) y la estructura interna hexagonal de cada servicio.
- Presentar el modelo de datos de nueve esquemas y sus reglas de integridad, incluidas las referencias entre servicios que no usan llaves foráneas.
- Documentar los mecanismos de comunicación síncrona (REST) y asíncrona (RabbitMQ), con el catálogo de eventos, las claves de enrutamiento y el sobre de mensajes.
- Registrar los endpoints cuyo contrato está documentado, sus parámetros y sus respuestas.
- Explicar el modelo de seguridad: autenticación con JWT, autorización por rol y propiedad, validaciones y protección de datos.
- Dejar recomendaciones de mantenimiento, evolución y pruebas, y señalar las decisiones pendientes y las discrepancias de la documentación.

6. Alcance técnico

6.1 Qué cubre este manual

- Los seis servicios de backend (security, customer, booking, operations, payment y notification) y la puerta de enlace.
- Las aplicaciones cliente (Angular para web y React Native con Expo para Android) únicamente en su relación con el backend: autenticación, consumo de la API y notificaciones push.
- La base de datos SQL Server de nueve esquemas, sus migraciones con Liquibase y sus mecanismos de auditoría.
- La mensajería con RabbitMQ y el servicio de correo SMTP para la recuperación de contraseña.
- El entorno de desarrollo local con Docker.

6.2 Qué no cubre este manual

- El uso de la plataforma por parte de clientes, operarios y administradores (Manual de usuario y Manual de administrador).
- El procedimiento paso a paso de instalación (Manual de instalación); aquí solo se describen los componentes y la configuración que la arquitectura exige.
- Los entornos de pruebas integradas (staging) y producción: la documentación los declara fuera del alcance de la versión académica.
- La integración con pasarelas de pago, la facturación electrónica y la gestión de inventarios, que no forman parte del sistema.

7. Descripción general del sistema

7.1 Resumen de la solución

El sistema atiende un solo establecimiento. Tres roles fijos interactúan con él: el cliente, que registra sus vehículos, reserva turnos, paga y califica; el operario, que registra su disponibilidad y ejecuta los servicios asignados; y el administrador, que opera el negocio, asigna operarios, revisa pagos y consulta reportes. El pago no usa pasarela externa: el cliente transfiere desde su propia aplicación bancaria y la plataforma recibe el comprobante, que el administrador aprueba o rechaza manualmente (ADR-008). No se recolectan ni almacenan datos de tarjetas.

7.2 Tipo de aplicación y componentes principales

Se trata de un sistema distribuido de tipo cliente-servidor con un backend de microservicios. La Tabla 2 resume los componentes que se ejecutan en el entorno local.

Tabla 2
Componentes principales del sistema

Componente	Tipo	Responsabilidad
Aplicación web	Cliente (Angular)	Interfaz de clientes y administradores.
Aplicación Android	Cliente (React Native / Expo)	Interfaz móvil de clientes y operarios; recibe notificaciones push.
API Gateway	Infraestructura (Spring Cloud Gateway)	Punto único de entrada externo, validación de JWT en el borde, correlación de peticiones.
security-service	Servicio de dominio	Personas, cuentas, roles, sesiones, recuperación de contraseña, preferencias y auditoría.
customer-service	Servicio de dominio	Clientes, tipos de vehículo, vehículos del cliente y copia de visualización de puntos.
booking-service	Servicio de dominio	Catálogo, precios, horario del establecimiento, bahías y reservas.
operations-service	Servicio de dominio	Operarios, disponibilidad, ausencias, asignación, ejecución y calificación.
payment-service	Servicio de dominio	Promociones, cuentas de pago, pagos, comprobantes y libro de puntos de fidelidad.
notification-service	Servicio de dominio	Notificaciones dentro del sistema, tokens de dispositivo y envío push.
SQL Server	Base de datos	Una base física con nueve esquemas lógicos.
RabbitMQ	Mensajería	Eventos de dominio asíncronos.
Mailpit / SMTP	Correo	Código de recuperación de contraseña y correo de bienvenida.

Nota. Elaboración propia con base en la arquitectura del sistema (ADR-009). El API Gateway es infraestructura y no se cuenta como séptimo servicio de dominio.

7.3 Restricciones arquitectónicas

- El MVP contempla un único establecimiento.
- SQL Server es el único motor de base de datos de aplicación en esta versión.
- Ningún servicio consulta directamente las tablas de otro como parte de su lógica de negocio.
- No existen llaves foráneas entre esquemas pertenecientes a servicios distintos.
- La verificación de pagos es manual; no se utiliza Wompi ni otra pasarela.
- El entorno de ejecución previsto es el local con Docker; no hay staging ni producción.

8. Tecnologías utilizadas

La Tabla 3 reúne las tecnologías que la documentación fija para cada componente. Las versiones precisas de las dependencias de cada servicio no se encuentran en el material analizado y deben tomarse de los archivos de construcción del código (por ejemplo, *pom.xml*), donde la política del proyecto exige fijar versiones exactas.

Tabla 3
Tecnologías y herramientas del proyecto

Componente	Tecnología	Versión / detalle
Lenguaje del backend	Java	JDK 21 LTS
Framework del backend	Spring Boot	3.x (Spring Web, Spring Security, Bean Validation)
Puerta de enlace	Spring Cloud Gateway	Punto de entrada externo
Persistencia	Spring Data JPA y NamedParameterJdbcTemplate	JPA en booking-service; consultas SQL con parámetros nombrados en otros adaptadores
Base de datos	Microsoft SQL Server	2016 o superior; una base, nueve esquemas
Migraciones	Liquibase	Un único changelog maestro ordenado para toda la base de datos, con archivos SQL con formato Liquibase
Mensajería	RabbitMQ	Exchange de tipo topic <i>carwash.events</i>
Correo en desarrollo	Mailpit	Receptor SMTP local
Notificaciones push	Expo Push API	Reenvío a FCM y APNs; requiere compilación EAS en el móvil
Frontend web	Angular	Cliente de administrador y cliente
Frontend móvil	React Native con Expo	Android; Expo Go no recibe push remoto
Contenedores	Docker y Docker Compose	Docker 24+, Compose 2.20+
Pruebas	JUnit 5, Mockito, AssertJ, Testcontainers	Pruebas unitarias, de integración y de contrato
Control de versiones	Git y GitHub	Convenciones en 00-governance/git-conventions.md
Integración continua	Jenkins	Configuración en la sección 10-devops del repositorio
Diagramas	C4 (Mermaid) y DBML	Diagramas de contexto, contenedores y entidad-relación
Contratos de API	OpenAPI 3.0.3	Un archivo YAML por servicio; Redocly para visualizarlos

Nota. Elaboración propia con base en la guía del stack Java/Spring, las ADR y la estrategia de migraciones. Los números de puerto de los seis servicios no están fijados en la documentación y deben tomarse de la configuración

activa de Docker.

9. Arquitectura del sistema

9.1 Estilo arquitectónico

El backend adopta una arquitectura de microservicios con propiedad lógica de los datos. Un microservicio es una unidad desplegable de forma independiente, organizada alrededor de una capacidad del negocio y propietaria de sus datos (Newman, 2021). En este proyecto, los seis servicios comparten por razones académicas una misma instancia de SQL Server, pero cada esquema tiene exactamente un servicio propietario y ningún servicio escribe ni consulta el esquema de otro (ADR-003). Esta decisión conserva lo esencial del principio de una base de datos por servicio y permite que, en el futuro, un servicio se lleve sus esquemas a otra instancia cambiando únicamente su cadena de conexión.

La descomposición actual en seis servicios reemplazó a una versión anterior con más servicios, que separaba conceptos estrechamente relacionados y obligaba a numerosas llamadas entre servicios (ADR-002, reemplazada por ADR-009). El nuevo agrupamiento mantiene juntas las transacciones y los invariantes que dependen entre sí.

Tabla 4

Servicios, esquemas propietarios y profundidad del dominio

Servicio	Esquemas	Tablas	Profundidad del dominio
security-service	security, audit	10	Media: autenticación, roles, reglas de contraseña y preferencias
customer-service	customer	3	Media: propiedad de clientes y vehículos
booking-service	catalog, booking	12	Alta: conflictos de turno, ciclo de vida de la reserva, congelación de precios
operations-service	execution	5	Alta: disponibilidad, asignación, ejecución y calificación
payment-service	promotion, payment	12	Alta: revisión de pagos, promociones y fidelidad
notification-service	notification	2	Baja: creación de notificaciones y estado de lectura

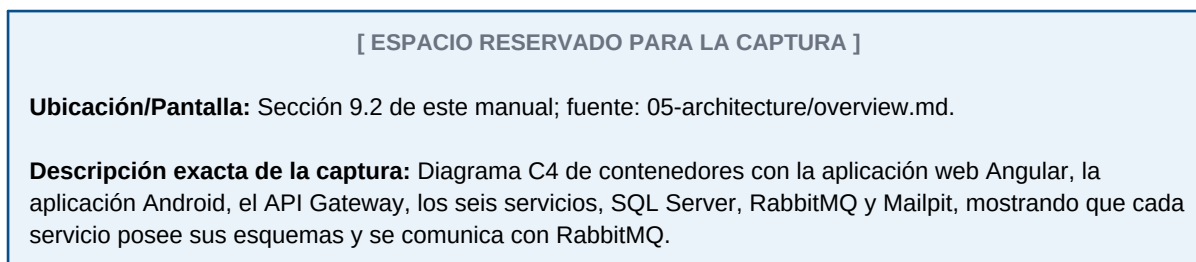
Nota. Fuente: ADR-009 y 06-data/models.md. El recuento de tablas corresponde al documento de modelos con fecha 02/10/2026; ver sección 11 y Anexo B.

Figura 1

Diagrama de contexto del sistema (C4, nivel 1)

[ESPACIO RESERVADO PARA LA CAPTURA]	
Ubicación/Pantalla: Sección 9.2 de este manual; fuente: 05-architecture/overview.md.	
Descripción exacta de la captura: Diagrama C4 de contexto con las tres personas (cliente, operario, administrador), el sistema Vehicle Washing en el centro y el sistema externo SMTP, con las relaciones etiquetadas (HTTPS y SMTP).	

Nota. Exportar el diagrama Mermaid del overview de arquitectura a imagen.

Figura 2**Diagrama de contenedores del sistema (C4, nivel 2)**

Nota. Debe verse con claridad quién publica y quién consume eventos.

9.2 Puerta de enlace (API Gateway)

Los clientes web y móvil llaman únicamente al API Gateway, de modo que no necesitan conocer las direcciones internas de los servicios (ADR-005). El gateway es infraestructura, no un servicio de dominio, y tiene las siguientes responsabilidades: enrutar las peticiones, rechazar en el borde los JWT ausentes, inválidos o vencidos, reenviar el encabezado Authorization, generar o propagar el identificador de correlación y aplicar los límites de petición o de archivo relevantes. No contiene reglas de negocio de reservas, pagos, asignación ni fidelidad. La verificación de autorización definitiva sigue en cada servicio.

Nota. Las plantillas del repositorio (catálogo de servicios y README del gateway) describen un límite de peticiones por IP y el uso de Redis. La decisión vigente (ADR-005 y reglas de seguridad) establece que no se aplica límite de peticiones en el gateway porque no hay exposición pública, y que Redis no se adopta. Este manual sigue la decisión vigente.

9.3 Arquitectura interna de cada servicio: hexagonal

Los seis servicios comparten el mismo estilo interno: arquitectura hexagonal, también llamada de puertos y adaptadores (Cockburn, 2005), con Java 21 y Spring Boot (ADR-007). Su objetivo es que las reglas de negocio no dependan de Spring, de JPA, de SQL Server ni de RabbitMQ. Las dependencias apuntan siempre hacia el dominio.

Tabla 5**Capas de la arquitectura hexagonal y su responsabilidad**

Capa	Contenido	Restricción
Dominio	Entidades, objetos de valor, invariantes, servicios de dominio, eventos y puertos de entrada y salida.	No importa <i>org.springframework.*</i> ni anotaciones JPA.
Aplicación	Casos de uso, orquestación y límites de transacción.	Coordina puertos; las reglas del negocio pertenecen al dominio.
Infraestructura	Controladores REST, persistencia (JPA/JDBC), mensajería RabbitMQ, almacenamiento de archivos y configuración.	Implementa los puertos de salida; los controladores son delgados.

Nota. Elaboración propia con base en ADR-007 y en el documento de arquitectura hexagonal.

Los puertos de entrada representan los casos de uso expuestos a los adaptadores; los puertos de salida son las interfaces que el dominio o la aplicación necesitan (repositorios, publicación de eventos, directorios de otros servicios). Las entidades de persistencia no se consideran entidades de dominio: un mapeador convierte entre ambas para mantener las columnas, los campos de auditoría y las anotaciones fuera del dominio. Un caso de uso típico sigue este patrón, ilustrado en la documentación del proyecto:

```
public final class ConfirmBookingUseCase {  
    public void execute(long bookingId) {  
        Booking booking = bookingRepository.findById(bookingId)  
            .orElseThrow(BookingNotFound::new);  
        booking.confirm();  
        bookingRepository.save(booking);  
        eventPublisher.publish(booking.confirmedEvent());  
    }  
}
```

Nota. Ejemplo ilustrativo tomado de la documentación de arquitectura hexagonal; no corresponde a una clase verificada del código entregado.

Las reglas de las pruebas se derivan de esta estructura: las reglas del dominio se prueban con JUnit sin iniciar Spring, mientras que las restricciones de base de datos, las transacciones, la mensajería y la concurrencia requieren pruebas de integración. El proyecto evita crear abstracciones únicamente para que el diagrama parezca hexagonal: el patrón se justifica cuando protege una regla de negocio o aísla una dependencia externa.

9.4 Comunicación entre componentes

9.4.1 Comunicación síncrona con REST

Se utiliza REST cuando quien llama necesita una respuesta para continuar (ADR-004). Ejemplos: validar que un vehículo pertenece al cliente antes de crear una reserva, validar el precio vigente de un servicio, comprobar la disponibilidad de un operario o validar una reserva antes de crear un pago. Las llamadas entre servicios no pasan por el gateway: quien llama reenvía el token del usuario autenticado y el identificador de correlación.

Tabla 6**Dependencias síncronas conocidas entre servicios**

Origen	Destino	Motivo
booking-service	customer-service	Resolver y validar la propiedad del vehículo y su tipo (ADR-012).
booking-service	operations-service	Validar disponibilidad cuando el flujo de reserva lo requiere.
payment-service	booking-service	Validar la reserva y el monto adeudado antes de crear o revisar el pago.
payment-service	customer-service	Validar información del cliente cuando una promoción o regla de fidelidad lo exige.
operations-service	security-service	Validar la identidad del usuario operario cuando se requiere.

Nota. Fuente: ADR-004 y ADR-012. Los contratos exactos de cada llamada pertenecen a los archivos OpenAPI de cada servicio.

9.4.2 Comunicación asíncrona con RabbitMQ

RabbitMQ se usa para el trabajo que puede ocurrir después de la transacción original: asignación tras la confirmación de una reserva, notificaciones, reacciones al estado de ejecución, aprobación de pagos y actualización de puntos. El detalle del exchange, las claves de enrutamiento y el sobre de los mensajes se encuentra en la sección 13.4.

Figura 3**Flujo principal de reserva, asignación, ejecución y notificación**

[ESPACIO RESERVADO PARA LA CAPTURA]	
Ubicación/Pantalla: Sección 9.4 de este manual; fuente: 02-domain/domain-events.md y 05-architecture/overview.md.	
Descripción exacta de la captura: Diagrama de secuencia o de flujo con las llamadas REST (booking hacia customer; payment hacia booking) y los eventos de RabbitMQ (booking.confirmed hacia operations y notification; execution.service_completed hacia notification; payment.confirmed hacia notification).	

9.5 Preocupaciones transversales**9.5.1 Auditoría y ciclo de vida de los datos**

Toda tabla de dominio lleva un bloque estándar de siete columnas de auditoría: *created_at*, *created_by*, *updated_at*, *updated_by*, *deleted_at*, *deleted_by* y *row_version*. Los datos de dominio nunca se borran físicamente: se usa borrado lógico mediante *deleted_at*. La tabla *audit.audit_log* registra los cambios mediante disparadores de base de datos; los servicios no publican eventos de auditoría como sustituto.

Para que el disparador conozca al autor del cambio, lee `SESSION_CONTEXT(N'user_id')`, que está vacío a menos que alguien lo establezca. Cada servicio debe ejecutar al inicio de cada petición, antes de tocar cualquier tabla, la instrucción siguiente con el identificador tomado del JWT autenticado:

```
EXEC sp_set_session_context @key = N'user_id', @value = @currentUserId;
```

Esto debe resolverse en un único lugar compartido por servicio (un filtro o interceptor) y no copiarse en cada caso de uso, porque olvidarlo deja *changed_by* en NULL sin generar ningún error visible. La documentación (ADR-009) lo clasifica como un riesgo real de probabilidad alta.

9.5.2 Concurrencia

La columna *row_version* implementa bloqueo optimista. Una actualización incluye la versión leída por la aplicación; si otro cambio ya modificó la fila, la actualización no afecta ninguna fila y la aplicación informa un conflicto. Un disparador generado (*tr_touch_<tabla>*) incrementa *row_version* y actualiza *updated_at* en cada UPDATE, de modo que el código de aplicación nunca los establece.

9.5.3 Transacciones

Cada caso de uso abre una transacción local, que solo puede escribir en varios esquemas si todos pertenecen al mismo servicio. No se usan transacciones entre servicios ni se mantiene una transacción abierta esperando a otro servicio. Una reserva y un pago no forman una sola transacción porque pertenecen a servicios distintos: su consistencia se logra con validación de la aplicación y eventos. Los eventos se publican después de confirmar la transacción local.

9.5.4 Registro, correlación y manejo de errores

Los servicios escriben sus registros en la salida estándar para consultarlos con Docker. Cada entrada debe incluir marca de tiempo, nivel, nombre del servicio e identificador de correlación. Nunca se registran contraseñas, JWT, tokens de recuperación, contenido de comprobantes ni datos personales innecesarios. El gateway genera o acepta el identificador de correlación y lo reenvía en las llamadas HTTP entre servicios y en los mensajes de RabbitMQ (clave MDC fijada por un filtro de correlación). El proyecto no incluye una plataforma de trazado distribuido.

La validación ocurre en tres capas: validación de entrada en el adaptador REST (código 400), validación de referencias entre servicios antes de escribir y reglas de dominio dentro del dominio o la aplicación (código 422).

Tabla 7**Categorías de respuesta HTTP usadas por la arquitectura**

Código	Significado
400	Petición o entrada inválida.
401	Autenticación ausente, inválida o vencida.
403	Autenticado pero sin autorización.
404	Recurso no encontrado.
409	Conflicto con el estado actual (por ejemplo, versión desactualizada).
422	Recurso válido pero operación inválida según las reglas de negocio.
500	Error inesperado del servidor.
503	Dependencia no disponible (por ejemplo, <i>CUSTOMER_SERVICE_UNAVAILABLE</i>).

Nota. Fuente: cross-cutting.md y ADR-012. Los códigos de error deben ser estables y no exponer trazas de pila ni información sensible.

9.5.5 Tiempo e idioma

Todas las marcas de tiempo se almacenan en UTC con el tipo *DATETIME2(3)* y se escriben con *SYSUTCDATETIME()*; los clientes las convierten a la hora de Colombia para mostrarlas. El backend debe devolver códigos estables de error y de evento en lugar de traducciones; el texto de las notificaciones se almacena ya redactado en español, junto con un código de tipo estable (ADR-011).

9.5.6 Patrones no adoptados

El proyecto decidió expresamente no adoptar, en esta versión, Outbox, Saga, CQRS, Event Sourcing, BFF, malla de servicios, caché Redis, base de datos por servicio, interruptor de circuito formal ni pasarela de pagos. Antes de incorporar un patrón nuevo, la guía de patrones exige responder qué problema concreto resuelve, qué requisito lo necesita, qué complejidad añade y si el equipo puede operarlo en el entorno local. La consecuencia más importante es la ausencia de Outbox: un evento puede perderse si el sistema falla entre la confirmación de la transacción y la publicación, riesgo aceptado en ADR-004.

10. Estructura del proyecto

Esta sección describe tres niveles de estructura: el repositorio de documentación que se analizó, la estructura interna que la arquitectura prescribe para cada servicio y los artefactos de código que la documentación nombra de forma explícita. Como se indicó en la sección 4.1, el material analizado no incluye el código fuente de los servicios; por ello no se presenta un árbol de carpetas inventado.

10.1 Repositorio de documentación

El repositorio sigue una organización por temas numerados, en la que cada carpeta responde a una pregunta del proceso de desarrollo.

Tabla 8

Carpetas del repositorio de documentación y su función

Carpeta	Función
00-governance	Gobierno: reglas de documentación, convenciones de Git y ágiles, definiciones de Listo y Terminado, políticas de seguridad.
01-context	Contexto: visión general, alcance y glosario del negocio.
02-domain	Modelo de dominio: mapa de contextos, entidades y reglas, eventos de dominio.
03-product	Producto.
04-requirements	Requisitos funcionales y no funcionales, historias de usuario y matriz de trazabilidad.
05-architecture	Arquitectura: visión general, hexagonal, guía de patrones, transversales, despliegue y registros de decisiones (ADR).
06-data	Datos: modelos por servicio, diccionario, convenciones, estrategia de migración y evaluación de normalización.
07-api	Contratos de API: guía, autenticación y archivos OpenAPI.
08-uml	Diagramas fuente (DBML y C4) y sus exportaciones.
09-microservices	Catálogo de servicios y documentación por servicio.
10-devops	Entornos y configuración local.
11-quality	Estrategia de pruebas y guía de TDD.
12-ux-ui	Sistema de diseño, mapa de navegación y maquetas HTML.
13-operations a 15-project-control	Operación, capacitación y control del proyecto.
_stacks	Guías por pila tecnológica (Java/Spring Boot, Node, Python, Go).
99-archive	Material histórico reemplazado.

Nota. Elaboración propia a partir del contenido del repositorio de documentación analizado.

10.2 Estructura interna prescrita para cada servicio

La arquitectura hexagonal fija la siguiente organización de paquetes para los seis servicios. Los nombres exactos pueden variar cuando el desarrollo se materializa, pero la dirección de las dependencias es obligatoria (hexagonal-architecture.md).

```
src/main/java/.../
|-- domain/
|   |-- model/           (entidades, objetos de valor, invariantes)
|   |-- event/           (eventos de dominio)
|   |-- service/         (servicios de dominio)
|   |-- port/
|       |-- in/           (casos de uso expuestos)
|       |-- out/          (repositorios, publicador de eventos, directorios)
|-- application/
|   |-- usecase/         (orquestración y transacciones)
|-- infrastructure/
|   |-- adapter/in/web/   (controladores y DTO REST)
|   |-- adapter/out/persistence/ (JPA/JDBC y mapeadores)
|   |-- adapter/out/messaging/ (RabbitMQ)
|   |-- adapter/out/storage/ (archivos)
|   |-- config/          (configuración de Spring)
```

Nota. La guía del stack Java/Spring utiliza una variante equivalente (*web*, *persistence*, *messaging*, *config* bajo *infrastructure*). Ambas respetan la misma regla de dependencia.

Tabla 9
Función de cada paquete

Paquete	Función
domain/model	Contiene entidades como Booking, Operator o Payment y objetos de valor como Money y TimeSlot. Sin dependencias de framework.
domain/port/in	Interfaces de casos de uso (por ejemplo, CreateBookingUseCase) que los adaptadores de entrada invocan.
domain/port/out	Interfaces que el dominio necesita: BookingRepository, EventPublisher, directorios de otros servicios.
application/usecase	Implementa los casos de uso, abre la transacción local y coordina los puertos.
infrastructure/adapter/in/web	Controladores delgados que validan la forma de la petición y llaman al puerto de entrada.
infrastructure/adapter/out/persistence	Implementa los repositorios con Spring Data JPA o JDBC y mapea entre entidades de persistencia y de dominio.
infrastructure/adapter/out/messaging	Serializa y publica eventos en RabbitMQ después de confirmar la transacción; los consumidores traducen mensajes a comandos.
infrastructure/config	Cableado de dependencias y configuración de seguridad, mensajería y propiedades.

Nota. Elaboración propia con base en la arquitectura hexagonal y la guía del stack Java/Spring.

10.3 Artefactos de código mencionados por la documentación

La documentación nombra de manera explícita los siguientes artefactos. Su ubicación exacta debe confirmarse en el código.

Tabla 10

Artefactos de código citados en la documentación

Artefacto	Servicio	Función según la documentación
platform-commons (CurrentUser, PlatformSecurityConfig)	Compartido	Extrae el usuario del JWT verificado y configura la seguridad: todo endpoint exige autenticación por defecto y las rutas públicas se declaran explícitamente.
CorrelationIdFilter	Compartido	Fija la clave MDC del identificador de correlación para registros y llamadas salientes.
CustomerDirectory (puerto) y CustomerServiceDirectory (adaptador REST)	booking-service	Resuelve vehículos llamando a customer-service con el token del usuario.
VehicleSnapshot	booking-service	Registro de dominio propio de booking que representa un vehículo; evita importar código de customer-service.
CustomerVehicleController, AdminVehicleController, LookupVehiclesUseCase	customer-service	Exponen los endpoints de vehículos consumidos por booking, incluido el de consulta administrativa por lote.
Changelogs de Liquibase 005, 016, 017 y 018; semillas 103, 104 y 109	booking y notification	Recreación de catalog y booking; tablas de notificación; estado de bahía y descanso; semillas de catálogo, estados y horarios.
lavarapido-infra/.env.example	Infraestructura	Nombres de variables de entorno (por ejemplo, CUSTOMER_SERVICE_URL, CUSTOMER_SERVICE_TIMEOUT, BOOKING_ZONE).

Nota. Fuente: ADR-012, ADR-010, ADR-011, reglas de seguridad y estrategia de migraciones.

11. Base de datos

El sistema utiliza una única instancia de Microsoft SQL Server 2016 o superior, con una base de datos dividida en nueve esquemas que pertenecen a seis servicios. El documento de modelos (fecha 02/10/2026) describe 44 tablas. Las ADR-010 y ADR-011 incorporan además las tablas *security.user_consent*, *security.person_address*, *execution.service_reassignment*, *notification.device_token* y *notification.processed_event*, que la propia ADR-011 indica que deben reflejarse en la documentación del modelo. La aplicación de ADR-009 también refiere 43 tablas en el documento de arquitectura. Estas diferencias se registran en el Anexo B; el equipo debe confirmar el recuento real contra los changelogs de Liquibase.

11.1 Diagrama entidad-relación

El diagrama entidad-relación completo y uno por servicio existen como archivos DBML en la carpeta 08-uml del repositorio (modelo completo y un archivo por servicio). Las referencias entre servicios se dibujan como comentarios, no como relaciones, porque no tienen llave foránea.

Figura 4

Diagrama entidad-relación completo del sistema

[ESPACIO RESERVADO PARA LA CAPTURA]

Ubicación/Pantalla: Carpeta 08-uml del repositorio, archivo del modelo completo (DBML).

Descripción exacta de la captura: Diagrama con las 44 tablas agrupadas por esquema (security, audit, customer, catalog, booking, execution, promotion, payment, notification), con llaves primarias, llaves foráneas dentro de cada servicio y anotaciones de las referencias entre servicios.

Nota. Exportar desde dbdiagram.io u otra herramienta compatible con DBML.

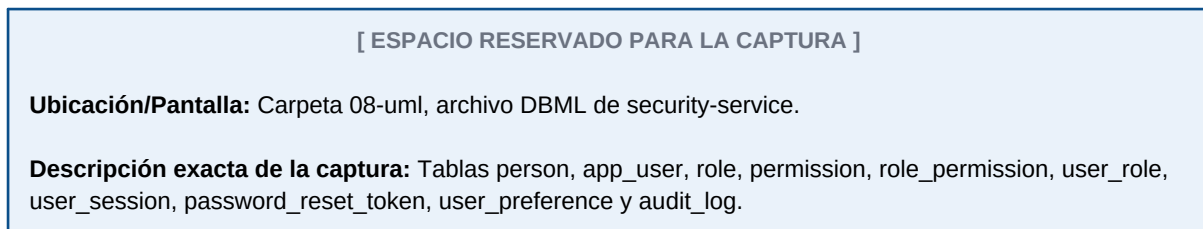
Figura 5

Diagrama entidad-relación del esquema booking y catalog

[ESPACIO RESERVADO PARA LA CAPTURA]

Ubicación/Pantalla: Carpeta 08-uml, archivo DBML de booking-service.

Descripción exacta de la captura: Tablas establishment, business_hour, business_hour_exception, bay_status, service_bay, booking_status, cancellation_reason, booking, booking_service, service_category, service y service_price con sus relaciones.

Figura 6**Diagrama entidad-relación del esquema security**

11.2 Tablas principales

11.2.1 Propiedad de esquemas y reglas de llaves foráneas

Las llaves foráneas se permiten solo dentro del límite de propiedad de un servicio. Entre servicios, las referencias son columnas con índice cuyo valor valida la aplicación antes de escribir (ADR-003). Por ejemplo, *booking.booking_service.service_price_id* sí es una llave foránea hacia *catalog.service_price* porque ambos esquemas pertenecen a booking-service, mientras que *payment.payment.booking_id* no lo es porque payment y booking pertenecen a servicios distintos.

Tabla 11**Referencias entre servicios sin llave foránea**

Columna	Apunta a	Propietario del destino
customer.customer.person_id	security.person	security-service
catalog.service_price.vehicle_type_id	customer.vehicle_type	customer-service
booking.booking.customer_vehicle_id	customer.customer_vehicle	customer-service
booking.booking.booked_by	security.app_user	security-service
execution.operator.user_id	security.app_user	security-service
execution.service_execution.booking_service_id	booking.booking_service	booking-service
promotion.promotion_service.service_id	catalog.service	booking-service
promotion.promotion_customer.customer_id	customer.customer	customer-service
promotion.promotion_customer.assigned_by	security.app_user	security-service
promotion.booking_promotion.booking_id	booking.booking	booking-service
payment.payment.booking_id	booking.booking	booking-service
payment.payment.approved_by	security.app_user	security-service
payment.payment_receipt.uploaded_by	security.app_user	security-service
payment.payment_receipt.reviewed_by	security.app_user	security-service
payment.loyalty_transaction.customer_id	customer.customer	customer-service
payment.loyalty_transaction.booking_id	booking.booking	booking-service
notification.notification.user_id	security.app_user	security-service

Nota. Fuente: sección "Cross-service contract" de 06-data/models.md. Son 17 columnas con índice. Las dos únicas con llave foránea entre esquemas son *security.user_preference.user_id* y *booking.booking_service.service_price_id*, ambas dentro de un mismo servicio.

11.2.2 Convenciones de modelado

- **Llaves sustitutas:** IDENTITY(1,1) entero (BIGINT en tablas de alto crecimiento como booking, payment y loyalty_transaction; INT o SMALLINT en catálogos), no UUID, para coincidir con el núcleo de seguridad compartido, evitar fragmentación del índice agrupado y mantener estables los identificadores sembrados de los que dependen algunas restricciones.
- **Bloque de auditoría:** siete columnas en toda tabla de dominio (excepto el núcleo de seguridad compartido y password_reset_token).
- **Borrado lógico:** ningún DELETE físico. Toda consulta debe incluir *deleted_at IS NULL* salvo que consulte el historial. Las unicidades de negocio (por ejemplo, la placa del vehículo o el par servicio-tipo de vehículo del precio) usan índices filtrados con esa condición.

- **Tiempo:** DATETIME2(3) en UTC.
- **Nombres:** esquemas de una palabra; tablas en snake_case y singular; llaves primarias *[tabla]_id*; restricciones *pk_*, *fk_*, *uq_*, *ck_*; índices *ix_* y *ux_*; disparadores *tr_* y *tr_touch_*. Los identificadores *[role]*, *[permission]* y *[action]* se escriben entre corchetes por ser palabras reservadas de T-SQL.
- **Catálogos:** once tablas de catálogo comparten la forma *code*, *name*, *display_order*, *is_active*. Se mantienen separadas (no una tabla genérica de estados) para que la llave foránea impida que una reserva adopte, por ejemplo, un estado de pago.
- **Códigos de estado:** la ADR-010 fija códigos en inglés en la base de datos (SCHEDULED, CONFIRMED, IN_PROGRESS, COMPLETED, CANCELLED, NO_SHOW) y el frontend los traduce. El documento de entidades y reglas aún lista códigos en español (AGENDADA, CONFIRMADA, etc.); ver Anexo B.

11.2.3 Descripción de las tablas por esquema

Esquema security (propietario: security-service).

Tabla 12

Tablas del esquema security

Tabla	Propósito	PK	Llaves foráneas internas
person	Datos de identidad de cualquier persona conocida por el sistema (número de documento único).	person_id	—
app_user	Credenciales de acceso; una cuenta por persona.	user_id	person_id -> security.person.person_id
role	Los tres roles fijos del sistema.	role_id	—
permission	Permisos granulares (recurso, acción); heredados del núcleo compartido.	permission_id	—
role_permission	Relación muchos a muchos entre roles y permisos.	role_permission_id	role_id -> security.role.role_id; permission_id -> security.permission.permission_id
user_role	Relación muchos a muchos entre usuarios y roles.	user_role_id	user_id -> security.app_user.user_id; role_id -> security.role.role_id
user_session	Sesiones abiertas: quién ingresó, cuándo y si se cerró.	user_session_id	user_id -> security.app_user.user_id
password_reset_token	Tokens de recuperación de contraseña con vencimiento y uso único.	password_reset_token_id	user_id -> security.app_user.user_id
user_preference	Preferencias de interfaz: tema, idioma y notificaciones.	user_preference_id	user_id -> Unique

Esquema audit (propietario: security-service).

Tabla 13

Tablas del esquema audit

Tabla	Propósito	PK	Llaves foráneas internas
audit_log	Historial único de cambios de todo el sistema, alimentado por disparadores.	audit_log_id	—

Esquema customer (propietario: customer-service).

Tabla 14**Tablas del esquema customer**

Tabla	Propósito	PK	Llaves foráneas internas
vehicle_type	Tipos de vehículo (moto, automóvil, camioneta, camión) con factor de tamaño.	vehicle_type_id	—
customer	Registro del cliente; vincula a la persona y guarda la copia de visualización de puntos.	customer_id	—
customer_vehicle	Vehículos del cliente.	customer_vehicle_id	customer_id -> customer.customer_id; vehicle_type_id -> customer.vehicle_type.vehicle_type_id

Esquema catalog (propietario: booking-service).**Tabla 15****Tablas del esquema catalog**

Tabla	Propósito	PK	Llaves foráneas internas
service_category	Agrupar los servicios para la pantalla del cliente.	service_category_id	—
service	Servicio ofrecido por el establecimiento.	service_id	service_category_id -> catalog.service_category.service_category_id
service_price	Precio y duración de un servicio para un tipo de vehículo, con vigencia.	service_price_id	service_id -> catalog.service.service_id

Esquema booking (propietario: booking-service).

Tabla 16
Tablas del esquema booking

Tabla	Propósito	PK	Llaves foráneas internas
establishment	Datos propios del negocio (encabezado de recibos, logotipo, contacto).	establishment_id	—
business_hour	Horario de atención por día de la semana, con descanso opcional.	business_hour_id	—
business_hour_exception	Festivos, cierres y horarios especiales.	business_hour_exception_id	—
bay_status	Catálogo de estados de una bahía (ACTIVE, MAINTENANCE, INACTIVE).	bay_status_id	—
service_bay	Bahías físicas de lavado; su cantidad es la capacidad del negocio.	service_bay_id	bay_status_id -> booking.bay_status.bay_status_id
booking_status	Catálogo de los seis estados de una reserva.	booking_status_id	—
cancellation_reason	Motivos de cancelación de una reserva.	cancellation_reason_id	—
booking	La reserva: núcleo del sistema.	booking_id	service_bay_id -> booking.service_bay.service_bay_id; booking_status_id -> booking.booking_status.booking_status_id; cancellation_reason_id ->
booking_service	Líneas de servicio incluidas en una reserva.	booking_service_id	booking_id -> booking.booking.booking_id; service_price_id -> catalog.service_price.service_price_id

Esquema execution (propietario: operations-service).

Tabla 17**Tablas del esquema execution**

Tabla	Propósito	PK	Llaves foráneas internas
execution_status	Estados de ejecución, distintos de los estados de reserva.	execution_status_id	—
operator	Operario (personal de lavado).	operator_id	—
operator_availability	Horario semanal de trabajo por operario.	operator_availability_id	operator_id -> execution.operator.operator_id
operator_absence	Ausencias puntuales que reemplazan la disponibilidad semanal.	operator_absence_id	operator_id -> execution.operator.operator_id
service_execution	Ejecución real de una línea de reserva, con operario y calificación.	service_execution_id	operator_id -> execution.operator.operator_id; execution_status_id -> execution.execution_status.execution_status_id

Esquema promotion (propietario: payment-service).**Tabla 18****Tablas del esquema promotion**

Tabla	Propósito	PK	Llaves foráneas internas
discount_type	Tipo de descuento: porcentaje o monto fijo.	discount_type_id	—
promotion	Promoción: código, descuento, elegibilidad y límites de uso.	promotion_id	discount_type_id -> promotion.discount_type.discount_type_id
promotion_service	Restringe una promoción a servicios específicos.	promotion_service_id	promotion_id -> promotion.promotion_id
promotion_customer	Asigna una promoción directamente a un cliente.	promotion_customer_id	promotion_id -> promotion.promotion_id
booking_promotion	Promoción aplicada a una reserva y descuento efectivo.	booking_promotion_id	promotion_id -> promotion.promotion_id

Esquema payment (propietario: payment-service).

Tabla 19

Tablas del esquema payment

Tabla	Propósito	PK	Llaves foráneas internas
payment_method_type	Efectivo, Nequi, Daviplata y transferencia bancaria.	payment_method_type_id	—
payment_account	Una fila por método de pago; aquí reside el código QR del negocio.	payment_account_id	payment_method_type_id -> payment.payment_method_type.payment_method_type_id
payment_status	Los cinco estados de un pago.	payment_status_id	—
payment	Un intento de pago para una reserva.	payment_id	payment_account_id -> payment.payment_account_id; payment_status_id -> payment.payment_status_id
payment_receipt	Comprobante que carga el cliente y su revisión.	payment_receipt_id	payment_id -> payment.payment_id
loyalty_movement_type	Los cuatro tipos de movimiento de puntos.	loyalty_movement_type_id	—
loyalty_transaction	Libro de puntos de fidelidad: cada movimiento en orden.	loyalty_transaction_id	loyalty_movement_type_id -> customer.loyalty_movement_type.loyalty_movement_type_id

Esquema notification (propietario: notification-service).

Tabla 20

Tablas del esquema notification

Tabla	Propósito	PK	Llaves foráneas internas
notification_type	Los tipos de evento que generan notificación.	notification_type_id	—
notification	Notificación enviada a un usuario.	notification_id	notification_type_id -> notification.notification_type_id

Nota. Elaboración propia a partir de 06-data/models.md. Las llaves foráneas listadas son las internas a cada servicio; las referencias entre servicios se encuentran en la Tabla 11.

11.3 Diccionario de datos

El diccionario siguiente se generó a partir del documento de modelos y lista, para cada tabla, sus columnas, tipos, nulabilidad y restricciones. Para mantenerlo legible, el bloque estándar de auditoría (siete columnas) se resume en una sola fila en las tablas que lo incluyen. La descripción

detallada de cada columna, el DDL completo y las decisiones de modelado de cada tabla se encuentran en 06-data/models.md y 06-data/data-dictionary.md.

11.3.1 Esquema security

security.person — Datos de identidad de cualquier persona conocida por el sistema (número de documento único).

Campo	Tipo	Nulo	Restricción
person_id	BIGINT	No	PK
document_number	NVARCHAR(20)	No	Unique
first_name	NVARCHAR(60)	No	—
last_name	NVARCHAR(60)	No	—
phone	NVARCHAR(20)	Yes	—
email	NVARCHAR(120)	Yes	—

security.app_user — Credenciales de acceso; una cuenta por persona.

Campo	Tipo	Nulo	Restricción
user_id	BIGINT	No	PK
person_id	BIGINT	No	FK -> security.person.person_id, Unique
username	NVARCHAR(60)	No	Unique
password_hash	NVARCHAR(255)	No	—
is_active	BIT	No	—
last_login	DATETIME2(3)	Yes	—

security.role — Los tres roles fijos del sistema.

Campo	Tipo	Nulo	Restricción
role_id	SMALLINT	No	PK
code	NVARCHAR(30)	No	Unique
name	NVARCHAR(60)	No	—

security.permission — Permisos granulares (recurso, acción); heredados del núcleo compartido.

Campo	Tipo	Nulo	Restricción
permission_id	SMALLINT	No	PK
code	NVARCHAR(60)	No	Unique
name	NVARCHAR(80)	No	—
resource	NVARCHAR(60)	No	—
action	NVARCHAR(30)	No	—

security.role_permission — Relación muchos a muchos entre roles y permisos.

Campo	Tipo	Nulo	Restricción
role_permission_id	BIGINT	No	PK
role_id	SMALLINT	No	FK -> security.role.role_id
permission_id	SMALLINT	No	FK -> security.permission.permission_id

security.user_role — Relación muchos a muchos entre usuarios y roles.

Campo	Tipo	Nulo	Restricción
user_role_id	BIGINT	No	PK
user_id	BIGINT	No	FK -> security.app_user.user_id
role_id	SMALLINT	No	FK -> security.role.role_id

security.user_session — Sesiones abiertas: quién ingresó, cuándo y si se cerró.

Campo	Tipo	Nulo	Restricción
user_session_id	BIGINT	No	PK
user_id	BIGINT	No	FK -> security.app_user.user_id
refresh_token_hash	NVARCHAR(255)	Yes	—
started_at	DATETIME2(3)	No	—
expires_at	DATETIME2(3)	No	—
revoked_at	DATETIME2(3)	Yes	—
ip_address	NVARCHAR(45)	Yes	—
user_agent	NVARCHAR(300)	Yes	—

security.password_reset_token — Tokens de recuperación de contraseña con vencimiento y uso único.

Campo	Tipo	Nulo	Restricción
password_reset_token_id	BIGINT	No	PK
user_id	BIGINT	No	FK -> security.app_user.user_id
token_hash	NVARCHAR(255)	No	Unique
requested_at	DATETIME2(3)	No	—
expires_at	DATETIME2(3)	No	—
used_at	DATETIME2(3)	Yes	—
ip_address	NVARCHAR(45)	Yes	—

security.user_preference — Preferencias de interfaz: tema, idioma y notificaciones.

Campo	Tipo	Nulo	Restricción
user_preference_id	BIGINT	No	PK
user_id	BIGINT	No	Unique, FK -> security.app_user.user_id
theme	NVARCHAR(20)	No	—
language	NVARCHAR(5)	No	ck_pref_language
notifications_enabled	BIT	No	—
created_at ... row_version	—	—	Bloque estándar de auditoría (siete columnas)

11.3.2 Esquema audit

audit.audit_log — Historial único de cambios de todo el sistema, alimentado por disparadores.

Campo	Tipo	Nulo	Restricción
audit_log_id	BIGINT	No	PK
entity	NVARCHAR(60)	No	—
entity_id	BIGINT	No	—
action	NVARCHAR(20)	No	—
previous_data	NVARCHAR(MAX)	Yes	—
new_data	NVARCHAR(MAX)	Yes	—
changed_by	BIGINT	Yes	—
changed_at	DATETIME2(3)	No	—

11.3.3 Esquema customer

customer.vehicle_type — Tipos de vehículo (moto, automóvil, camioneta, camión) con factor de tamaño.

Campo	Tipo	Nulo	Restricción
vehicle_type_id	SMALLINT	No	PK
code	NVARCHAR(30)	No	Unique
name	NVARCHAR(60)	No	—
size_factor	DECIMAL(4,2)	No	—
display_order	SMALLINT	No	—
is_active	BIT	No	—
created_at ... row_version	—	—	Bloque estándar de auditoría (siete columnas)

customer.customer — Registro del cliente; vincula a la persona y guarda la copia de visualización de puntos.

Campo	Tipo	Nulo	Restricción
customer_id	BIGINT	No	PK
person_id	BIGINT	No	Unique, Sin FK (referencia entre servicios)
loyalty_points	INT	No	—
customer_since	DATE	No	—
created_at ... row_version	—	—	Bloque estándar de auditoría (siete columnas)

customer.customer_vehicle — Vehículos del cliente.

Campo	Tipo	Nulo	Restricción
customer_vehicle_id	BIGINT	No	PK
customer_id	BIGINT	No	FK -> customer.customer.customer_id
license_plate	NVARCHAR(10)	No	—
vehicle_type_id	SMALLINT	No	FK -> customer.vehicle_type.vehicle_type_id
brand	NVARCHAR(50)	Yes	—
color	NVARCHAR(30)	Yes	—
created_at ... row_version	—	—	Bloque estándar de auditoría (siete columnas)

11.3.4 Esquema catalog

catalog.service_category — Agrupa los servicios para la pantalla del cliente.

Campo	Tipo	Nulo	Restricción
service_category_id	SMALLINT	No	PK
code	NVARCHAR(30)	No	Unique
name	NVARCHAR(60)	No	—
description	NVARCHAR(200)	Yes	—
display_order	SMALLINT	No	—
is_active	BIT	No	—
created_at ... row_version	—	—	Bloque estándar de auditoría (siete columnas)

catalog.service — Servicio ofrecido por el establecimiento.

Campo	Tipo	Nulo	Restricción
service_id	INT	No	PK

Campo	Tipo	Nulo	Restricción
code	NVARCHAR(30)	No	Unique
name	NVARCHAR(100)	No	—
description	NVARCHAR(MAX)	Yes	—
service_category_id	SMALLINT	No	FK -> catalog.service_category.service_category_id
is_active	BIT	No	—
created_at ... row_version	—	—	Bloque estándar de auditoría (siete columnas)

catalog.service_price — Precio y duración de un servicio para un tipo de vehículo, con vigencia.

Campo	Tipo	Nulo	Restricción
service_price_id	BIGINT	No	PK
service_id	INT	No	FK -> catalog.service.service_id
vehicle_type_id	SMALLINT	No	Sin FK (referencia entre servicios)
price	DECIMAL(12,2)	No	—
estimated_minutes	SMALLINT	No	—
valid_from	DATE	No	—
valid_to	DATE	Yes	—
created_at ... row_version	—	—	Bloque estándar de auditoría (siete columnas)

11.3.5 Esquema booking

booking.establishment — Datos propios del negocio (encabezado de recibos, logotipo, contacto).

Campo	Tipo	Nulo	Restricción
establishment_id	SMALLINT	No	PK
legal_name	NVARCHAR(150)	No	—
trade_name	NVARCHAR(150)	No	—
tax_id	NVARCHAR(30)	No	—
address	NVARCHAR(200)	No	—
phone	NVARCHAR(20)	Yes	—
email	NVARCHAR(120)	Yes	—
logo_url	NVARCHAR(500)	Yes	—
singleton	BIT	No	Unique

booking.business_hour — Horario de atención por día de la semana, con descanso opcional.

Campo	Tipo	Nulo	Restricción
business_hour_id	SMALLINT	No	PK
day_of_week	SMALLINT	No	Unique
opens_at	TIME(0)	No	—
closes_at	TIME(0)	No	—
break_starts_at	TIME(0)	Yes	—
break_ends_at	TIME(0)	Yes	—
is_active	BIT	No	—
created_at ... row_version	—	—	Bloque estándar de auditoría (siete columnas)

booking.business_hour_exception — Festivos, cierres y horarios especiales.

Campo	Tipo	Nulo	Restricción
business_hour_exception_id	INT	No	PK
exception_date	DATE	No	Unique
is_closed	BIT	No	—
opens_at	TIME(0)	Yes	—
closes_at	TIME(0)	Yes	—
reason	NVARCHAR(120)	No	—
created_at ... row_version	—	—	Bloque estándar de auditoría (siete columnas)

booking.bay_status — Catálogo de estados de una bahía (ACTIVE, MAINTENANCE, INACTIVE).

Campo	Tipo	Nulo	Restricción
bay_status_id	SMALLINT	No	PK
code	NVARCHAR(30)	No	Unique
name	NVARCHAR(60)	No	—
accepts_bookings	BIT	No	—
display_order	SMALLINT	No	—
created_at ... row_version	—	—	Bloque estándar de auditoría (siete columnas)

booking.service_bay — Bahías físicas de lavado; su cantidad es la capacidad del negocio.

Campo	Tipo	Nulo	Restricción
service_bay_id	SMALLINT	No	PK

Campo	Tipo	Nulo	Restricción
code	NVARCHAR(20)	No	Unique
name	NVARCHAR(60)	No	—
bay_status_id	SMALLINT	No	FK -> booking.bay_status.bay_status_id
created_at ... row_version	—	—	Bloque estándar de auditoría (siete columnas)

booking.booking_status — Catálogo de los seis estados de una reserva.

Campo	Tipo	Nulo	Restricción
booking_status_id	SMALLINT	No	PK
code	NVARCHAR(30)	No	Unique
name	NVARCHAR(60)	No	—
is_final	BIT	No	—
display_order	SMALLINT	No	—
is_active	BIT	No	—
created_at ... row_version	—	—	Bloque estándar de auditoría (siete columnas)

booking.cancellation_reason — Motivos de cancelación de una reserva.

Campo	Tipo	Nulo	Restricción
cancellation_reason_id	SMALLINT	No	PK
code	NVARCHAR(30)	No	Unique
name	NVARCHAR(60)	No	—
is_customer_fault	BIT	No	—
display_order	SMALLINT	No	—
is_active	BIT	No	—
created_at ... row_version	—	—	Bloque estándar de auditoría (siete columnas)

booking.booking — La reserva: núcleo del sistema.

Campo	Tipo	Nulo	Restricción
booking_id	BIGINT	No	PK
customer_vehicle_id	BIGINT	No	Sin FK (referencia entre servicios)
service_bay_id	SMALLINT	Yes	FK -> booking.service_bay.service_bay_id
scheduled_start	DATETIME2(3)	No	—
scheduled_end	DATETIME2(3)	No	—

Campo	Tipo	Nulo	Restricción
booking_status_id	SMALLINT	No	FK -> booking.booking_status.booking_status_id
booked_by	BIGINT	No	Sin FK (referencia entre servicios)
cancellation_reason_id	SMALLINT	Yes	FK -> booking.cancellation_reason.cancellation_reason_id
points_redeemed	INT	No	—
points_discount_amount	DECIMAL(12,2)	No	—
notes	NVARCHAR(300)	Yes	—
created_at ... row_version	—	—	Bloque estándar de auditoría (siete columnas)

booking.booking_service — Líneas de servicio incluidas en una reserva.

Campo	Tipo	Nulo	Restricción
booking_service_id	BIGINT	No	PK
booking_id	BIGINT	No	FK -> booking.booking.booking_id
service_price_id	BIGINT	No	FK -> catalog.service_price.service_price_id
quantity	SMALLINT	No	—
created_at ... row_version	—	—	Bloque estándar de auditoría (siete columnas)

11.3.6 Esquema execution

execution.execution_status — Estados de ejecución, distintos de los estados de reserva.

Campo	Tipo	Nulo	Restricción
execution_status_id	SMALLINT	No	PK
code	NVARCHAR(30)	No	Unique
name	NVARCHAR(60)	No	—
is_final	BIT	No	—
display_order	SMALLINT	No	—
is_active	BIT	No	—
created_at ... row_version	—	—	Bloque estándar de auditoría (siete columnas)

execution.operator — Operario (personal de lavado).

Campo	Tipo	Nulo	Restricción
operator_id	INT	No	PK
user_id	BIGINT	No	Unique, Sin FK (referencia entre servicios)
hired_on	DATE	No	—
is_active	BIT	No	—
created_at ... row_version	—	—	Bloque estándar de auditoría (siete columnas)

execution.operator_availability — Horario semanal de trabajo por operario.

Campo	Tipo	Nulo	Restricción
operator_availability_id	INT	No	PK
operator_id	INT	No	FK -> execution.operator.operator_id
day_of_week	SMALLINT	No	—
starts_at	TIME(0)	No	—
ends_at	TIME(0)	No	—
is_active	BIT	No	—
created_at ... row_version	—	—	Bloque estándar de auditoría (siete columnas)

execution.operator_absence — Ausencias puntuales que reemplazan la disponibilidad semanal.

Campo	Tipo	Nulo	Restricción
operator_absence_id	INT	No	PK
operator_id	INT	No	FK -> execution.operator.operator_id
starts_at	DATETIME2(3)	No	—
ends_at	DATETIME2(3)	No	—
reason	NVARCHAR(120)	No	—
created_at ... row_version	—	—	Bloque estándar de auditoría (siete columnas)

execution.service_execution — Ejecución real de una línea de reserva, con operario y calificación.

Campo	Tipo	Nulo	Restricción
service_execution_id	BIGINT	No	PK
booking_service_id	BIGINT	No	Unique, Sin FK (referencia entre servicios)
operator_id	INT	No	FK -> execution.operator.operator_id
execution_status_id	SMALLINT	No	FK -> execution.execution_status.execution_status_id

Campo	Tipo	Nulo	Restricción
started_at	DATETIME2(3)	Yes	—
finished_at	DATETIME2(3)	Yes	—
quality_rating	SMALLINT	Yes	—
quality_comment	NVARCHAR(500)	Yes	ck_sexec_comment
rated_at	DATETIME2(3)	Yes	—
is_comment_visible	BIT	No	—
created_at ... row_version	—	—	Bloque estándar de auditoría (siete columnas)

11.3.7 Esquema promotion

promotion.discount_type — Tipo de descuento: porcentaje o monto fijo.

Campo	Tipo	Nulo	Restricción
discount_type_id	SMALLINT	No	PK
code	NVARCHAR(30)	No	Unique
name	NVARCHAR(60)	No	—
display_order	SMALLINT	No	—
is_active	BIT	No	—
created_at ... row_version	—	—	Bloque estándar de auditoría (siete columnas)

promotion.promotion — Promoción: código, descuento, elegibilidad y límites de uso.

Campo	Tipo	Nulo	Restricción
promotion_id	INT	No	PK
code	NVARCHAR(30)	No	Unique
name	NVARCHAR(100)	No	—
description	NVARCHAR(300)	Yes	—
discount_type_id	SMALLINT	No	FK -> promotion.discount_type.discount_type_id
discount_value	DECIMAL(12,2)	No	—
max_discount_amount	DECIMAL(12,2)	Yes	—
min_purchase_amount	DECIMAL(12,2)	No	—
is_public	BIT	No	—
min_completed_booking	INT	No	—

Campo	Tipo	Nulo	Restricción
valid_from	DATE	No	—
valid_to	DATE	No	—
max_redemptions	INT	Yes	—
max_redemptions_per_customer	INT	Yes	—
is_active	BIT	No	—
created_at ... row_version	—	—	Bloque estándar de auditoría (siete columnas)

promotion.promotion_service — Restringe una promoción a servicios específicos.

Campo	Tipo	Nulo	Restricción
promotion_service_id	INT	No	PK
promotion_id	INT	No	FK -> promotion.promotion.promotion_id
service_id	INT	No	Sin FK (referencia entre servicios)
created_at ... row_version	—	—	Bloque estándar de auditoría (siete columnas)

promotion.promotion_customer — Asigna una promoción directamente a un cliente.

Campo	Tipo	Nulo	Restricción
promotion_customer_id	BIGINT	No	PK
promotion_id	INT	No	FK -> promotion.promotion.promotion_id
customer_id	BIGINT	No	Sin FK (referencia entre servicios)
assigned_at	DATETIME2(3)	No	—
assigned_by	BIGINT	Yes	Sin FK (referencia entre servicios)
created_at ... row_version	—	—	Bloque estándar de auditoría (siete columnas)

promotion.booking_promotion — Promoción aplicada a una reserva y descuento efectivo.

Campo	Tipo	Nulo	Restricción
booking_promotion_id	BIGINT	No	PK
booking_id	BIGINT	No	Sin FK (referencia entre servicios)
promotion_id	INT	No	FK -> promotion.promotion.promotion_id
applied_amount	DECIMAL(12,2)	No	—
created_at ... row_version	—	—	Bloque estándar de auditoría (siete columnas)

11.3.8 Esquema payment

payment.payment_method_type — Efectivo, Nequi, Daviplata y transferencia bancaria.

Campo	Tipo	Nulo	Restricción
payment_method_type_id	SMALLINT	No	PK
code	NVARCHAR(30)	No	Unique
name	NVARCHAR(60)	No	—
requires_account	BIT	No	—
requires_receipt	BIT	No	—
display_order	SMALLINT	No	—
is_active	BIT	No	—
created_at ... row_version	—	—	Bloque estándar de auditoría (siete columnas)

payment.payment_account — Una fila por método de pago; aquí reside el código QR del negocio.

Campo	Tipo	Nulo	Restricción
payment_account_id	SMALLINT	No	PK
payment_method_type_id	SMALLINT	No	FK -> payment.payment_method_type.payment_method_type_id
account_holder	NVARCHAR(120)	No	—
account_number	NVARCHAR(50)	Yes	—
qr_image_url	NVARCHAR(500)	Yes	—
instructions	NVARCHAR(300)	Yes	—
is_active	BIT	No	—
created_at ... row_version	—	—	Bloque estándar de auditoría (siete columnas)

payment.payment_status — Los cinco estados de un pago.

Campo	Tipo	Nulo	Restricción
payment_status_id	SMALLINT	No	PK
code	NVARCHAR(30)	No	Unique
name	NVARCHAR(60)	No	—
is_final	BIT	No	—
display_order	SMALLINT	No	—
is_active	BIT	No	—

Campo	Tipo	Nulo	Restricción
created_at ... row_version	—	—	Bloque estándar de auditoría (siete columnas)

payment.payment — Un intento de pago para una reserva.

Campo	Tipo	Nulo	Restricción
payment_id	BIGINT	No	PK
booking_id	BIGINT	No	Sin FK (referencia entre servicios)
payment_account_id	SMALLINT	No	FK -> payment.payment_account.payment_account_id
payment_status_id	SMALLINT	No	FK -> payment.payment_status.payment_status_id
amount	DECIMAL(12,2)	No	—
processed_at	DATETIME2(3)	Yes	—
approved_by	BIGINT	Yes	Sin FK (referencia entre servicios)
rejection_reason	NVARCHAR(200)	Yes	—
created_at ... row_version	—	—	Bloque estándar de auditoría (siete columnas)

payment.payment_receipt — Comprobante que carga el cliente y su revisión.

Campo	Tipo	Nulo	Restricción
payment_receipt_id	BIGINT	No	PK
payment_id	BIGINT	No	FK -> payment.payment.payment_id
file_url	NVARCHAR(500)	No	—
transaction_reference	NVARCHAR(100)	Yes	—
reported_amount	DECIMAL(12,2)	Yes	—
uploaded_at	DATETIME2(3)	No	—
uploaded_by	BIGINT	No	Sin FK (referencia entre servicios)
reviewed_at	DATETIME2(3)	Yes	—
reviewed_by	BIGINT	Yes	Sin FK (referencia entre servicios)
review_comment	NVARCHAR(300)	Yes	—
created_at ... row_version	—	—	Bloque estándar de auditoría (siete columnas)

payment.loyalty_movement_type — Los cuatro tipos de movimiento de puntos.

Campo	Tipo	Nulo	Restricción
loyalty_movement_type_id	SMALLINT	No	PK
code	NVARCHAR(30)	No	Unique
name	NVARCHAR(60)	No	—
sign	SMALLINT	No	—
display_order	SMALLINT	No	—
is_active	BIT	No	—
created_at ... row_version	—	—	Bloque estándar de auditoría (siete columnas)

payment.loyalty_transaction — Libro de puntos de fidelidad: cada movimiento en orden.

Campo	Tipo	Nulo	Restricción
loyalty_transaction_id	BIGINT	No	PK
customer_id	BIGINT	No	Sin FK (referencia entre servicios)
loyalty_movement_type_id	SMALLINT	No	FK -> customer.loyalty_movement_type.loyalty_movement_type_id
booking_id	BIGINT	Yes	Sin FK (referencia entre servicios)
points	INT	No	—
balance_after	INT	No	—
expires_on	DATE	Yes	—
description	NVARCHAR(200)	Yes	—
created_at ... row_version	—	—	Bloque estándar de auditoría (siete columnas)

11.3.9 Esquema notification

notification.notification_type — Los tipos de evento que generan notificación.

Campo	Tipo	Nulo	Restricción
notification_type_id	SMALLINT	No	PK
code	NVARCHAR(40)	No	Unique
name	NVARCHAR(80)	No	—
display_order	SMALLINT	No	—
is_active	BIT	No	—
created_at ... row_version	—	—	Bloque estándar de auditoría (siete columnas)

notification.notification — Notificación enviada a un usuario.

Campo	Tipo	Nulo	Restricción
notification_id	BIGINT	No	PK
user_id	BIGINT	No	Sin FK (referencia entre servicios)
notification_type_id	SMALLINT	No	FK -> notification.notification_type.notification_type_id
title	NVARCHAR(120)	No	—
message	NVARCHAR(500)	No	—
reference_entity	NVARCHAR(40)	Yes	—
reference_id	BIGINT	Yes	—
is_read	BIT	No	—
read_at	DATETIME2(3)	Yes	—
sent_at	DATETIME2(3)	No	—
created_at ... row_version	—	—	Bloque estándar de auditoría (siete columnas)

11.4 Reglas de integridad que la base de datos no puede imponer

Algunas reglas de negocio no pueden expresarse como restricciones de base de datos y deben garantizarse en la aplicación. La Tabla siguiente las reúne, junto con los mecanismos de base de datos que sí existen.

Tabla 21

Reglas de integridad impuestas por la aplicación

Regla	Dónde se garantiza
Una línea de servicio solo se califica cuando su ejecución está finalizada, una sola vez y por el cliente dueño de la reserva (INV-007). La calificación es opcional.	operations-service (aplicación)
<i>booking.points_discount_amount</i> debe coincidir con el movimiento REDEEMED de la reserva en <i>payment.loyalty_transaction</i> .	payment-service tras validar el saldo
<i>customer.loyalty_points</i> es una copia de visualización del último <i>balance_after</i> del libro; las redenciones se validan contra el libro, nunca contra la copia.	payment-service
Una reserva no puede quedar fuera del horario (<i>business_hour</i>), considerando las excepciones (<i>business_hour_exception</i>).	booking-service
El operario asignado debe estar disponible según <i>operator_availability</i> y sin ausencia superpuesta en <i>operator_absence</i> .	operations-service
Dos reservas activas no pueden ocupar la misma bahía en rangos de tiempo superpuestos.	Disparadores de superposición (changeset 011) y aplicación
El precio de una línea de reserva no cambia si cambia el catálogo (inmutabilidad del precio).	Disparador de inmutabilidad de <i>service_price</i> (changeset 012) y aplicación
Un pago aprobado por reserva como máximo (índice único <i>ux_payment_one_approved_per_booking</i>); una referencia de transacción no puede reutilizarse en otro pago aprobado.	Índice único y aplicación
<i>booking</i> no almacena total: se deriva de las líneas y sus precios menos el descuento por puntos.	Diseño (ADR-010)

Nota. Fuente: models.md, entities-and-rules.md, migration-strategy.md y ADR-008/ADR-010.

11.5 Migraciones y datos iniciales

Las migraciones se gestionan con Liquibase usando changelogs en SQL con formato Liquibase (*--liquibase formatted sql* y un *--changeset* por unidad de cambio). La estrategia vigente define **un único changelog ordenado para toda la base de datos**, no uno por servicio: el archivo raíz *db/changelog/db.changelog-master.xml* incluye cada archivo en el orden exacto de ejecución, porque Liquibase respeta el orden de la lista y no el alfabético. Liquibase registra lo ejecutado en *DATABASECHANGELOG* y coordina despliegues concurrentes con *DATABASECHANGELOGLOCK*. Los changesets son siempre hacia adelante, sin bloques de reversión. La documentación no indica qué proceso ejecuta el changelog maestro (un paso de despliegue dedicado o el arranque de un servicio); ver Anexo B.

Tabla 22
Orden de los changelogs de la estrategia de migración

Archivo	Contenido
001-create-schemas	Los nueve esquemas y un rol de base de datos por servicio con derechos solo sobre sus esquemas.
002-create-security-and-audit-core	person, app_user, role, permission, role_permission, user_role, user_session, password_reset_token y audit_log.
003 a 010	user_preference y los esquemas customer, catalog, booking, execution, promotion, payment y notification.
011-create-overlap-triggers	Disparadores contra superposición de turnos.
012-create-service-price-immutability-trigger	Inmutabilidad del precio.
013-create-audit-triggers	Generación de <i>tr_touch_<tabla></i> para toda tabla con <i>row_version</i> ; debe ejecutarse al final.
seed/101 a 109	Datos iniciales: roles, catálogos de vehículo, categorías de servicio, estados de reserva, estados de ejecución, tipos de descuento, catálogos de pago, tipos de notificación, bahías y horarios.

Nota. Fuente: 06-data/migration-strategy.md. La numeración puede haber evolucionado (por ejemplo, ADR-011 cita los changesets 016 y 017). Una tabla de dominio nueva, creada después del changeset 013, debe crear explícitamente su disparador *tr_touch_*, porque la generación masiva ya no se vuelve a ejecutar.

12. Módulos técnicos

Cada módulo técnico corresponde a un servicio. Para cada uno se indican su función, sus tablas, las reglas de negocio que protege, las dependencias y los eventos que publica o consume. Los nombres de archivos específicos se incluyen solo cuando la documentación los menciona; en el resto, la relación entre módulo, tabla y regla es la fuente confiable.

12.1 security-service

Función: Identidad y acceso: personas, cuentas, roles fijos, sesiones, recuperación de contraseña por código, preferencias de usuario y auditoría. Emite el JWT.

Tablas: security.person, app_user, role, permission, role_permission, user_role, user_session, password_reset_token, user_preference, y audit.audit_log (más user_consent y person_address según ADR-010).

Reglas principales: Las contraseñas nunca se guardan en texto plano (BCrypt). Una cuenta se bloquea 15 minutos tras 5 intentos fallidos en 15 minutos (contadores en memoria, se reinician al reiniciar el servicio). Las cuentas se desactivan, no se eliminan, y al desactivarlas se cierran sus sesiones. El correo (usuario de ingreso) se cambia confirmando con la contraseña actual. El código de recuperación dura 15 minutos, se invalida al usarse y su número de intentos está limitado.

Dependencias: SMTP (Mailpit en local) para el código de recuperación y el correo de bienvenida; RabbitMQ para publicar eventos cuando MESSAGING_ENABLED está activo.

Eventos: Publica UserRegistered (security.user_registered) y UserAuthenticated (security.user_authenticated).

12.2 customer-service

Función: Clientes, tipos de vehículo y vehículos de cada cliente.

Tablas: customer.vehicle_type, customer, customer_vehicle (con el atributo model añadido por ADR-010).

Reglas principales: Un vehículo pertenece a un solo cliente y solo su dueño lo gestiona. Una reserva solo usa vehículos del cliente que la crea. Desactivar un vehículo no borra el historial. La placa es única entre registros no eliminados (índice filtrado). El saldo de puntos de este servicio es una copia de visualización.

Dependencias: Es consultado por booking-service por REST (vehículo por identificador, vehículos propios y consulta administrativa por lote).

Endpoints documentados: GET /api/v1/vehicles/{id}, GET /api/v1/vehicles, GET /api/v1/admin/vehicles?ids= (ver sección 13).

12.3 booking-service

Función: Catálogo, precios por tipo de vehículo, datos y horario del establecimiento, bahías y reservas.

Tablas: catalog.service_category, service, service_price; booking.establishment, business_hour, business_hour_exception, bay_status, service_bay, booking_status, cancellation_reason, booking, booking_service.

Reglas principales: Una reserva exige un vehículo válido del cliente, al menos una línea de servicio y un rango donde el fin sea posterior al inicio. No puede confirmarse si el turno no cabe en el horario y las bahías disponibles. Dos reservas activas no ocupan una bahía superpuesta. El precio de cada línea es el vigente al crear la línea y no se reescribe. La modificación y la cancelación se restringen según el ciclo de vida (bloqueadas cuando la ejecución comenzó). El canje de puntos registra a la vez los puntos y su descuento monetario. Ante un turno ocupado se proponen hasta cinco horarios alternativos del mismo día.

Dependencias: customer-service por REST (puerto CustomerDirectory); si falla (tiempo agotado, 5xx o cuerpo no interpretable) responde 503 CUSTOMER_SERVICE_UNAVAILABLE; un vehículo ajeno o inexistente responde 404 VEHICLE_NOT_FOUND.

Eventos: Publica booking.created, booking.confirmed, booking.modified y booking.cancelled.

12.4 operations-service

Función: Operarios, disponibilidad, ausencias, asignación, ejecución de cada línea y calificación.

Tablas: execution.execution_status, operator (con specialty), operator_availability, operator_absence, service_execution, service_reassignment (ADR-010).

Reglas principales: Un operario solo se asigna si está disponible y no se superpone con otro servicio. Solo el operario asignado puede iniciar o finalizar su ejecución. Cada línea de reserva tiene como máximo una ejecución. La calificación (1 a 5, con comentario opcional que exige puntaje) pertenece a la ejecución finalizada y se guarda en service_execution, sin tabla propia. La asignación puede ser automática (solo por disponibilidad), elegida por el cliente o definida por el administrador; las reasignaciones conservan historial.

Eventos: Publica execution.operator_assigned, execution.operator_reassigned, execution.service_started, execution.service_completed y execution.rating_submitted; consume booking.confirmed para iniciar la asignación.

12.5 payment-service

Función: Promociones, cuentas de pago, pagos, comprobantes y libro de puntos de fidelidad.

Tablas: promotion.discount_type, promotion, promotion_service, promotion_customer, booking_promotion; payment.payment_method_type, payment_account, payment_status, payment, payment_receipt, loyalty_movement_type, loyalty_transaction.

Reglas principales: Todo pago pertenece a una reserva. Los métodos digitales exigen comprobante, monto declarado y referencia; el efectivo lo confirma directamente el administrador. El monto del comprobante se compara con el adeudado y una diferencia se trata como motivo de

revisión o rechazo. Una referencia usada en un pago aprobado no puede reutilizarse. Una reserva no tiene más de un pago aprobado. No se almacenan datos de tarjeta. El libro de puntos es la fuente de verdad del saldo.

Dependencias: booking-service y customer-service por REST para validar reserva, monto y cliente.

Eventos: Publica payment.initiated, payment.confirmed y payment.rejected; los eventos de promoción y fidelidad aparecen como planificados.

12.6 notification-service

Función: Bandeja de notificaciones dentro del sistema, estado de lectura y envío push mediante la API de Expo.

Tablas: notification.notification_type, notification, device_token y processed_event (ADR-011).

Reglas principales: La notificación siempre se almacena primero; un fallo de push nunca hace fallar la petición. Un token de dispositivo es único y se reasigna si otra cuenta inicia sesión en el mismo teléfono; la respuesta DeviceNotRegistered de Expo lo desactiva. El consumidor registra cada eventId en processed_event dentro de la misma transacción de su trabajo, de modo que las entregas duplicadas no generan notificaciones duplicadas. Un evento sin identificador del destinatario no produce notificación.

Configuración: PUSH_ENABLED (por defecto false: solo registra), MESSAGING_ENABLED (por defecto false), MAIL_ENABLED y MAIL_*. La cola notification-service.events reintenta tres veces y luego envía a notification-service.events.dlq.

12.7 Ciclos de vida

El ciclo de la reserva, el de la ejecución y el del pago son las máquinas de estado centrales. La Tabla 23 las resume con los códigos de base de datos vigentes según ADR-010 y ADR-008.

Tabla 23**Máquinas de estado del dominio**

Entidad	Estados (códigos de base de datos)	Transiciones principales
Reserva (booking)	SCHEDULED, CONFIRMED, IN_PROGRESS, COMPLETED, CANCELLED, NO_SHOW	SCHEDULED -> CONFIRMED -> IN_PROGRESS -> COMPLETED; SCHEDULED o CONFIRMED -> CANCELLED; -> NO_SHOW. La cancelación se bloquea una vez comienza la ejecución.
Ejecución del servicio	PENDIENTE, EN_PROCESO, PAUSADA, FINALIZADA, CON_NOVEDAD	PENDIENTE -> EN_PROCESO -> FINALIZADA; EN_PROCESO <-> PAUSADA; cualquiera -> CON_NOVEDAD. Las transiciones exactas están sujetas a los requisitos operativos.
Pago	PENDING, IN_REVIEW, APPROVED, REJECTED, REFUNDED	PENDING -> IN_REVIEW -> APPROVED o REJECTED; APPROVED -> REFUNDED.

Nota. Los códigos de ejecución aún están en español en la documentación consultada; la ADR-010 solo modificó los de reserva. Ver Anexo B.

Figura 7**Diagrama de estados de la reserva**

[ESPACIO RESERVADO PARA LA CAPTURA]	
Ubicación/Pantalla:	Sección 12.7 de este manual; fuente: 02-domain/entities-and-rules.md.
Descripción exacta de la captura:	Diagrama de estados con SCHEDULED, CONFIRMED, IN_PROGRESS, COMPLETED, CANCELLED y NO_SHOW, y las flechas de transición rotuladas (confirm, start, finish, cancel, no-show).

13. API o rutas del sistema

La API externa sigue el estilo REST (Fielding, 2000): recursos identificados por rutas, operaciones expresadas con los métodos HTTP y representaciones en JSON. Todo acceso externo pasa por el API Gateway, que enruta hacia los seis servicios. Los contratos formales deben residir en archivos OpenAPI 3.0.3, uno por servicio, en la carpeta 07-api/contracts/openapi del repositorio.

Estado de los contratos. En el material analizado solo existen el contrato del gateway, el de un servicio de autenticación y una plantilla genérica. El contrato de autenticación describe una pila distinta de la vigente (RS256, refresh token de siete días, PostgreSQL, Redis e identificadores UUID) y es, en la práctica, una plantilla sin adaptar; contradice a la ADR-006. Por ello, este manual **no lo reproduce como contrato vigente** y se limita a los endpoints que las decisiones de arquitectura y las ADR citan de forma explícita (sección 13.3). El equipo debe completar y publicar los seis contratos OpenAPI reales (Anexo B).

13.1 Convenciones generales

- **Versionado:** el prefijo */api/v1* en la ruta.
- **Formato:** JSON en peticiones y respuestas; identificadores numéricos (llaves IDENTITY), no UUID.
- **Autenticación:** encabezado *Authorization: Bearer <JWT>* en toda ruta no pública.
- **Correlación:** encabezado *X-Correlation-Id*, generado o aceptado por el gateway y propagado entre servicios.
- **Ownership del recurso:** la identidad se extrae del JWT verificado (*CurrentUser*), nunca de datos enviados por el cliente. Los precios y la hora de fin de una reserva tampoco se aceptan del cliente.
- **Paginación:** los endpoints de consulta masiva deben paginar (regla de seguridad A04). La plantilla compartida propone los parámetros *page* y *limit* (máximo 100).
- **Errores:** la plantilla compartida define un objeto con *error* (código en SCREAMING_SNAKE_CASE), *message*, *details* opcional y *traceId*. Los códigos deben ser estables y no exponer trazas internas. La Tabla 7 resume las categorías de estado HTTP.

13.2 Autenticación en la API

El servicio de seguridad emite un JWT firmado con HS256 (secreto compartido entregado como variable de entorno) con vigencia de una hora y claims reducidos: *sub* (identificador del usuario), *pid* (identificador de la persona), *roles* y *sid* (identificador de sesión, usado al cerrar sesión). No se usa refresh token: al vencer, la persona vuelve a iniciar sesión. Un JWT (Jones et al., 2015) no puede revocarse antes de su vencimiento; este riesgo se acepta de forma explícita (sección 14.1). La gestión de la cuenta, la recuperación de contraseña y el cierre de sesión son operaciones del servicio de seguridad cuyos contratos exactos deben incorporarse a su archivo OpenAPI.

13.3 Endpoints documentados

La Tabla 24 lista los endpoints citados por las ADR y las políticas. Las rutas de ADR-010 se escriben como la documentación las cita (sin el prefijo de versión); deben confirmarse en el contrato.

Tabla 24
Endpoints con documentación explícita

Método	Ruta	Rol	Descripción
GET	/health (gateway)	Público	Estado del gateway y de sus servicios; responde ok o degraded, o 503 si un servicio crítico cae.
POST	/api/v1/auth/login	Público	Autentica y entrega el token de acceso (citado por el contrato del gateway). Bloqueo con 423 tras intentos fallidos según la plantilla.
POST	/api/v1/bookings	CLIENT	Crea una reserva. Cuerpo: customerVehicleId, scheduledStart, services[] (serviceld, quantity) y notes. Valida propiedad del vehículo y precios vigentes.
GET	/api/v1/bookings/me	CLIENT	Reservas del usuario autenticado (propiedad por <i>booked_by</i>).
POST	/api/v1/admin/bookings	ADMIN	Crea una reserva en nombre de cualquier cliente.
GET	/api/v1/admin/bookings	ADMIN	Lista reservas (por ejemplo, las de un día) con el propietario del vehículo resuelto.
GET	/api/v1/vehicles/{id}	CLIENT	Devuelve el vehículo del token; 404 si no existe o es de otro cliente.
GET	/api/v1/vehicles	CLIENT	Vehículos del usuario autenticado.
GET	/api/v1/admin/vehicles?ids={ids}	ADMIN	Consulta por lote (hasta 100 por bloque); devuelve OwnedVehicleResponse con <i>vehicle</i> y <i>ownerUserId</i> .
PUT	/users/me/email	Autenticado	Cambia el correo de ingreso; exige la contraseña actual.
POST	/users/me/deactivate	Autenticado	Desactiva la cuenta propia; exige la contraseña.
PATCH	/admin/users/{id}/status	ADMIN	Activa o desactiva una cuenta.
POST	/api/v1/admin/notifications	ADMIN	Envía un anuncio (tipo SYSTEM_MESSAGE) o prueba notificaciones.

Nota. Fuente: ADR-005, ADR-010, ADR-011, ADR-012, reglas de seguridad y contrato del gateway.

13.3.1 Creación de una reserva

El cuerpo de la petición valida su forma en el adaptador REST (error 400) y las reglas de negocio las valida el dominio (error 422). Las anotaciones de Bean Validation de la política de seguridad definen la forma esperada: el identificador del vehículo es obligatorio, la fecha de inicio debe ser futura, debe haber al menos un servicio, la cantidad mínima es uno y las notas admiten un máximo de 300 caracteres.

```
POST /api/v1/bookings
Authorization: Bearer <JWT>
Content-Type: application/json

{
  "customerVehicleId": 12,
  "scheduledStart": "2026-10-12T15:00:00Z",
  "services": [ { "serviceId": 1, "quantity": 1 } ],
  "notes": "Revisar tapicería"
}
```

Nota. Ejemplo construido a partir de la estructura CreateBookingRequest de la política de seguridad; los valores son ficticios. El formato de la respuesta debe tomarse del contrato OpenAPI de booking-service.

Tabla 25

Respuestas de error relevantes en la resolución del vehículo

Situación	Respuesta	Motivo
Vehículo inexistente o de otro cliente	404 VEHICLE_NOT_FOUND	Misma respuesta a propósito, para no revelar la existencia de vehículos ajenos.
customer-service no responde, devuelve 5xx o un cuerpo inválido	503 CUSTOMER_SERVICE_UNAVAILABLE	No es culpa del llamador; no debe reportarse como error 4xx de la reserva.
Regla de negocio incumplida (turno ocupado, fuera de horario)	422	El recurso es válido pero la operación no lo es.
Forma de petición inválida	400	Validación en el adaptador REST.

Nota. Fuente: ADR-012 y cross-cutting.md.

Figura 8

Documentación interactiva de la API (Swagger UI o Redocly)

[ESPACIO RESERVADO PARA LA CAPTURA]	
Ubicación/Pantalla:	Visor de OpenAPI del proyecto (por ejemplo, Redocly generado desde 07-api/contracts/openapi).
Descripción exacta de la captura:	Pantalla con la lista de endpoints de booking-service agrupados por etiqueta y el detalle expandido de POST /api/v1/bookings, con su esquema de petición y sus respuestas.

Nota. Tomar la captura solo cuando exista el contrato OpenAPI real del servicio.

13.4 Eventos de dominio (RabbitMQ)

La mensajería asíncrona usa un único exchange de tipo topic, *carwash.events*, compartido por los seis servicios. Cada servicio consumidor tiene una cola enlazada a las claves de enrutamiento que necesita. La convención de la clave es *<esquema-propietario>.<evento_en_snake_case>*, de modo que el prefijo indica a la vez a qué servicio pertenece el hecho (Hohpe y Woolf, 2003, describen el patrón de publicación y suscripción).

Tabla 26
Catálogo de eventos, claves de enrutamiento y publicador

Evento	Clave de enrutamiento	Publicador
UserRegistered	security.user_registered	security-service
UserAuthenticated	security.user_authenticated	security-service
BookingCreated	booking.created	booking-service
BookingConfirmed	booking.confirmed	booking-service
BookingModified	booking.modified	booking-service
BookingCancelled	booking.cancelled	booking-service
OperatorAssigned	execution.operator_assigned	operations-service
OperatorReassigned	execution.operator_reassigned	operations-service
ServiceStarted	execution.service_started	operations-service
ServiceCompleted	execution.service_completed	operations-service
RatingSubmitted	execution.rating_submitted	operations-service
PaymentInitiated	payment.initiated	payment-service
PaymentConfirmed	payment.confirmed	payment-service
PaymentRejected	payment.rejected	payment-service
PromotionApplied	payment.promotion_applied	payment-service (planificado)
LoyaltyPointsEarned	payment.loyalty_points_earned	payment-service (planificado)
LoyaltyPointsRedeemed	payment.loyalty_points_redeemed	payment-service (planificado)
NotificationCreated	notification.created	notification-service

Nota. Fuente: cross-cutting.md, sección 7. Los eventos de rating usan el prefijo execution porque el dato reside físicamente en service_execution.

Todo mensaje del exchange lleva el mismo sobre; solo cambia el contenido de *payload*:

```
{
  "eventId": "uuid (clave de idempotencia del consumidor)",
  "eventType": "BookingConfirmed",
  "aggregateId": "uuid del agregado que lo produjo",
  "occurredAt": "2026-10-12T15:00:00Z",
  "version": 1,
  "payload": { "...": "campos propios del evento" }
}
```

Tabla 27**Reglas de entrega y procesamiento de mensajes**

Aspecto	Regla vigente
Entrega	Al menos una vez: el consumidor debe tolerar duplicados.
Confirmación (ack)	Después de procesar con éxito.
Reintento	Reintentos limitados con espera creciente; en notification-service son tres.
Cola de mensajes fallidos	Los mensajes fallidos se conservan para revisión (notification-service.events.dlq).
Idempotencia	El consumidor compara el eventId con su registro de eventos procesados, en la misma transacción que su trabajo.
Publicación	Después de confirmar la transacción de base de datos.
Outbox	No adoptado en esta versión; un evento puede perderse si falla la publicación.

Nota. Fuente: cross-cutting.md y ADR-004. Todo evento nuevo debe tener un consumidor documentado y una razón para existir.

13.4.1 Contrato de notificaciones

Todo evento que debe producir una notificación incluye en su *payload* el identificador del usuario destinatario (*security.app_user*), porque notification-service no debe leer datos de otros servicios para resolverlo (ADR-011). Los campos que lee notification-service son: *userId*, *email* y *firstName* en UserRegistered; *customerUserId*, *bookingId*, *bookingCode* y *scheduledStart* (ISO 8601 UTC) en los eventos de reserva; *operatorUserId* en las asignaciones; y *customerUserId*, *paymentId* y *amount* en los eventos de pago (con *reason* en el rechazo). Los doce códigos de tipo son los once ligados a eventos más SYSTEM_MESSAGE; cada uno se asigna a una categoría de visualización (REMINDER, PROMOTION, CONFIRMATION, CANCELLATION, MESSAGE o SYSTEM) calculada en código, no almacenada.

13.5 Verificación y salud

Cada servicio debe exponer un endpoint de salud o de disponibilidad, y el gateway publica uno propio que agrega el estado de sus servicios. Las verificaciones de integración deben cubrir la conectividad con la base de datos y RabbitMQ, la autenticación JWT, las validaciones entre servicios, la creación de reservas y el manejo de conflictos, y la revisión de pagos.

14. Seguridad

La política de seguridad del proyecto se basa en cinco principios: defensa en profundidad, mínimo privilegio, falla segura, seguridad por diseño y confianza cero (se verifica siempre, incluso dentro de la red interna). Su lista de control sigue el estándar OWASP Top 10 (OWASP Foundation, 2021).

14.1 Autenticación y control de sesiones

Tabla 28

Parámetros de autenticación

Elemento	Decisión
Token	JWT Bearer emitido por security-service.
Algoritmo	HS256 con secreto compartido en variable de entorno (mínimo 32 caracteres). Compromiso consciente del entorno académico (riesgo AT-006); RS256 es la evolución prevista.
Vigencia	Una hora.
Claims	sub, pid, roles y sid; sin nombre, correo, teléfono ni documento.
Contraseñas	BCrypt (Provos y Mazières, 1999); nunca MD5 ni SHA-1.
Refresh token	No se usa en esta versión.
Bloqueo de cuenta	Cinco intentos fallidos en 15 minutos bloquean la cuenta 15 minutos; los contadores están en memoria y se reinician con el servicio. Cambiar la contraseña por recuperación levanta el bloqueo.
Sesiones	<code>security.user_session</code> registra quién ingresó y cuándo; no se consulta en cada petición.
Revocación	Un JWT no se revoca antes de vencer (riesgo AT-007): cerrar sesión cierra la fila de sesión y un bloqueo impide nuevos ingresos, pero un token ya emitido sigue válido hasta agotar su hora.

Nota. Fuente: ADR-006, security-policy.md y security-rules.md.

14.2 Autorización

El sistema tiene tres roles fijos, sin superadministrador ni roles personalizados. La base de datos almacena los códigos ADMIN, OPERATOR y CLIENT; los documentos de arquitectura los nombran Administrador, Operario y Cliente. Aunque el esquema de seguridad define permisos granulares (recurso y acción), los requisitos solo piden control a nivel de rol, de modo que el rol es la unidad de autorización hasta que se decida lo contrario.

Tabla 29

Roles y alcance de la autorización

Rol	Alcance principal
ADMIN	Configura el establecimiento, servicios y promociones; asigna y reasigna operarios; revisa y aprueba pagos; consulta reportes.
OPERATOR	Registra su disponibilidad; consulta y actualiza solo las ejecuciones que le fueron asignadas.
CLIENT	Gestiona sus vehículos y reservas, paga por QR, califica servicios finalizados y ve solo sus propios registros.

Nota. Fuente: security-policy.md.

Un rol válido no basta. Cada servicio verifica además la **propiedad**: un operario con token válido no actualiza una ejecución ajena y un cliente solo alcanza sus propias reservas, vehículos y pagos. Los identificadores secuenciales no se usan como único control de acceso: exponer un identificador adivinable no es lo mismo que autorizar su acceso. El gateway rechaza en el borde los tokens inválidos y, además, cada servicio vuelve a validar el JWT y declara el rol con `@PreAuthorize`, porque las llamadas entre servicios no pasan por el gateway. Cuando un servicio llama a otro reenvía el JWT del usuario, de modo que el servicio destino autoriza al usuario original y no al servicio que llama.

14.3 Validación de entrada

- Validar en el borde (controlador) con Bean Validation y rechazar pronto con 400; la validación de los formularios del cliente es solo una comodidad.
- Lista blanca en lugar de lista negra: se define lo permitido.
- **Inyección SQL**: se usan siempre consultas parametrizadas (parámetros nombrados en `NamedParameterJdbcTemplate` o JPA tipado); está prohibido concatenar cadenas en SQL.
- **XSS**: en Angular se usa la interpolación, que escapa por defecto, y no se evade el sanitizador con datos del usuario.
- El precio y la hora de fin de una reserva nunca provienen del cliente.

14.4 Cifrado, secretos y comunicación

- HTTPS obligatorio fuera del entorno local, con TLS 1.2 como mínimo y HSTS en producción.
- Los servicios se comunican por la red interna de Docker y solo el gateway queda expuesto; no se usa mTLS en esta versión.
- Los secretos (cadena de SQL Server, credenciales de RabbitMQ, secreto del JWT y SMTP) se leen de variables de entorno desde un archivo `.env` excluido de Git; el repositorio solo contiene `.env.example` con valores de ejemplo. Cada servicio usa un login de base de datos con derechos únicamente sobre sus esquemas.

- El secreto del JWT y las contraseñas de base de datos se cambian antes de cualquier despliegue compartido o público, y de inmediato ante sospecha de compromiso. Las credenciales de demostración de los datos semilla nunca llegan a un despliegue público.
- Nunca se registran contraseñas, tokens ni el contenido de los comprobantes.

14.5 Lista de control OWASP Top 10

Tabla 30

Controles implementados por categoría OWASP

Categoría	Control
A01 Control de acceso roto	Verificación de rol (@PreAuthorize) y de propiedad en cada servicio; la identidad sale del JWT, no del cliente.
A02 Fallos criptográficos	BCrypt, TLS 1.2+ fuera de local, secretos en variables de entorno.
A03 Inyección	Parámetros nombrados en todo SQL y Bean Validation en toda petición.
A04 Diseño inseguro	Revisión de privacidad de las historias que exponen datos; paginación obligatoria; identificadores no usados como control de acceso.
A05 Configuración incorrecta	Docker Compose como definición única de cada entorno; encabezados de seguridad de Spring Security; sin trazas de pila en producción.
A06 Componentes vulnerables	Revisión de dependencias antes de cada entrega con <code>mvn versions:display-dependency-updates</code> ; versiones fijas, sin versiones flotantes.
A07 Fallos de autenticación	JWT de una hora, sin refresh token, bloqueo tras cinco intentos fallidos.
A08 Integridad de software y datos	Versiones fijas de dependencias y validación de los mensajes de RabbitMQ antes de procesarlos.
A09 Fallos de registro	Registros sin datos personales y con identificador de correlación; cambios en <code>audit.audit_log</code> .
A10 SSRF	Lista blanca de URL externas; no seguir redirecciones automáticamente.

Nota. Fuente: `security-policy.md` y `security-rules.md`. No hay límite de peticiones en el gateway: la defensa contra fuerza bruta es el bloqueo de cuenta.

14.6 Protección de datos personales

El tratamiento de datos personales debe cumplir la Ley Estatutaria 1581 de 2012 (Congreso de la República de Colombia, 2012). Por esta razón, la ADR-010 incorpora la tabla `security.user_consent`, que registra el tipo de consentimiento, la versión del documento, la fecha de aceptación y la IP. No se recolectan ni almacenan datos de tarjetas (número, CVV o vencimiento). El único dato personal que circula por el bus de mensajes es el correo y el nombre en el evento `UserRegistered`, y su método `toString()` los omite para que no lleguen a los registros.

14.7 Auditoría y eventos de seguridad

Tabla 31**Eventos de seguridad y lugar de registro**

Evento	Dónde se registra
Ingreso exitoso	<i>security.user_session</i> (usuario, inicio, vencimiento, IP y agente) y <i>app_user.last_login</i> .
Ingreso fallido y bloqueo	Registro de <i>security-service</i> .
Cierre de sesión	<i>security.user_session.revoked_at</i> .
Cambio de contraseña	Registro de <i>security-service</i> y <i>audit.audit_log</i> .
Cambios de datos de dominio	<i>audit.audit_log</i> , mediante disparadores que leen el usuario de <i>SESSION_CONTEXT</i> .

Nota. Fuente: *security-policy.md* y *cross-cutting.md*.

14.8 Riesgos aceptados

- Imposibilidad de revocar un JWT antes de su vencimiento (AT-007).
- Firma simétrica HS256 con secreto compartido entre servicios (AT-006).
- Contadores de bloqueo en memoria: un reinicio del servicio los borra.
- Posible pérdida de eventos entre la confirmación y la publicación, por ausencia de Outbox.
- Menor aislamiento físico por compartir una instancia de SQL Server entre servicios.
- Sin límite de peticiones en el gateway, justificado por la ausencia de exposición pública.

15. Control de versiones

Esta sección trata del control de versiones del código y de los artefactos del proyecto; el historial de cambios del presente manual se encuentra al inicio del documento.

15.1 Código fuente

El proyecto usa Git con repositorios alojados en GitHub. Las convenciones de ramas y de mensajes de confirmación están definidas en 00-governance/git-conventions.md del repositorio de documentación. La plantilla de entorno local ejemplifica el flujo diario con la rama *develop* y mensajes con el formato *tipo(alcance): descripción*, por ejemplo *feat(scope): description*. Los repositorios mencionados por la documentación incluyen el de infraestructura (*lavarapido-infra*, con el *.env.example*) y los de cada servicio y cliente; sus direcciones deben consignarse en el catálogo de servicios, que aún las muestra como plantilla.

15.2 Esquema de base de datos

El esquema se versiona con Liquibase: cada cambio es un changeset en un archivo SQL con identificador, autor y ruta, y Liquibase calcula una suma de verificación de cada uno; si el contenido de un changeset ya ejecutado cambia, la herramienta falla. Por eso los changesets son inmutables y siempre hacia adelante: un cambio posterior se registra en un changeset nuevo.

15.3 Decisiones de arquitectura

Las decisiones se registran como ADR numeradas con estado (Propuesta, Aceptada, Reemplazada). La ADR-002 fue reemplazada por la ADR-009, y la ADR-010 enmendó el 30/09/2026 el cambio del correo electrónico. Toda decisión que modifique documentos existentes debe listar los que se deben actualizar.

Tabla 32
Registros de decisiones de arquitectura

ADR	Tema	Estado
ADR-002	Estilo arquitectónico y descomposición original	Reemplazada por ADR-009
ADR-003	Estrategia de base de datos	Aceptada
ADR-004	Comunicación entre servicios	Aceptada
ADR-005	API Gateway	Aceptada
ADR-006	Autenticación y autorización	Aceptada
ADR-007	Arquitectura interna de servicios	Aceptada
ADR-008	Verificación de pagos por QR y comprobante	Aceptada
ADR-009	Descomposición revisada en seis servicios	Propuesta (en uso)
ADR-010	Alineación del modelo de datos y el frontend	Aceptada
ADR-011	Notificaciones: push y contrato de eventos	Aceptada
ADR-012	Reserva: resolución del vehículo por REST	Aceptada

Nota. Fuente: carpeta de registros de decisiones. La ADR-001 se menciona como relacionada pero no se analizó.

16. Recomendaciones de mantenimiento

16.1 Entorno local y puesta en marcha

El único entorno en alcance es el local con Docker (Docker 24 o superior, Compose 2.20 o superior). Requiere SQL Server, RabbitMQ, Mailpit, los seis servicios, el gateway y los clientes. Los servicios no deben depender de un orden de arranque fijo más allá de la disponibilidad de la infraestructura: SQL Server y RabbitMQ deben estar listos antes que los servicios que los usan, y cada servicio debe realizar comprobaciones de disponibilidad en vez de suponer que otro contenedor está listo por haberse iniciado. Una secuencia práctica es SQL Server, RabbitMQ, Mailpit, security, customer, booking, operations, payment, notification y por último el gateway. Los puertos concretos de los servicios deben tomarse de la configuración de Docker activa.

16.2 Variables de entorno y configuración

Tabla 33

Variables de configuración citadas por la documentación

Variable	Efecto
CUSTOMER_SERVICE_URL, CUSTOMER_SERVICE_TIMEOUT	Dirección y tiempo de espera de la llamada de booking-service a customer-service.
BOOKING_ZONE	Zona horaria de las horas reservables.
MESSAGING_ENABLED	Activa RabbitMQ en security-service (publicador) y notification-service (consumidor); por defecto false.
PUSH_ENABLED	Activa el envío real a la API de Expo; por defecto false, solo se registra.
MAIL_ENABLED y MAIL_*	Correo de bienvenida y recuperación; Gmail con contraseña de aplicación o Mailpit.
Secreto del JWT	Clave HS256, mínimo 32 caracteres, igual en los servicios que validan el token.
Conexión a SQL Server y credenciales de RabbitMQ	Valores propios de cada entorno; nunca se versionan.

Nota. Los nombres exactos de las variables de conexión y del secreto deben tomarse del *.env.example* de lavarapido-infra; el *.env.example* de la documentación es una plantilla genérica (PostgreSQL y Redis) y no refleja el entorno real.

16.3 Actualizaciones y dependencias

- Ejecutar `mvn versions:display-dependency-updates` antes de cada entrega académica y corregir las vulnerabilidades críticas o altas antes de la siguiente sesión de revisión.
- Fijar versiones exactas en *pom.xml* o *build.gradle*; no usar versiones flotantes.
- Mantener actualizadas las ADR y los documentos de datos y de API cuando cambie el diseño; la ADR-010 y la ADR-011 ya señalan documentos pendientes de actualizar.

16.4 Base de datos, copias de seguridad y recuperación

- Cada esquema puede respaldarse, scriptarse o migrarse de forma independiente; conviene respaldar por esquema antes de cualquier migración.
- SQL Server usa almacenamiento persistente en Docker. La documentación no define una estrategia de respaldo de producción porque la producción está fuera del alcance.
- El almacenamiento de los comprobantes de pago sigue pendiente de decisión: no debe fijarse una ruta ni un formato hasta que se defina.
- Una tabla nueva creada después del changeset 013 debe crear su propio disparador *tr_touch_*.
- Con borrado lógico, toda consulta nueva debe incluir *deleted_at IS NULL*.

16.5 Pruebas

La estrategia de pruebas propone una pirámide con aproximadamente 50 % de pruebas unitarias, 25 % de integración, 20 % de contrato y 5 % de extremo a extremo, con una cobertura objetivo de al menos 80 % de líneas y de al menos 90 % en el dominio. Las pruebas unitarias del dominio y de los casos de uso no requieren Spring ni base de datos; las de integración usan SQL Server y RabbitMQ reales con Testcontainers o Docker; las de contrato verifican que se cumpla el contrato OpenAPI.

Pruebas de concurrencia recomendadas: dos reservas simultáneas para el mismo turno y bahía, dos actualizaciones con la misma *row_version*, y entrega duplicada del mismo evento. Son las situaciones donde fallan con mayor probabilidad las garantías descritas en este manual.

16.6 Diagnóstico de problemas frecuentes

Tabla 34**Problemas frecuentes y revisión sugerida**

Síntoma	Causa probable	Revisión
Campo <i>changed_by</i> en NULL en <i>audit.audit_log</i>	El servicio no estableció <i>SESSION_CONTEXT</i> al inicio de la petición.	Revisar el filtro o interceptor común del servicio.
503 <i>CUSTOMER_SERVICE_UNAVAILABLE</i> al reservar	<i>customer-service</i> caído o con tiempo de espera excedido.	Revisar <i>CUSTOMER_SERVICE_URL</i> y <i>CUSTOMER_SERVICE_TIMEOUT</i> y la salud del servicio.
Notificaciones que no llegan	<i>MESSAGING_ENABLED</i> en false, evento sin <i>customerUserId</i> o mensaje en la cola DLQ.	Revisar la configuración, el contenido del evento y la cola <i>notification-service.events.dlq</i> .
Push que no se entrega	<i>PUSH_ENABLED</i> en false, compilación sin EAS o token desactivado.	Revisar la configuración y la tabla <i>device_token</i> .
401 con un token que parecía válido	Token vencido (una hora) o secreto distinto entre servicios.	Volver a iniciar sesión y verificar que el secreto del JWT es igual en todos los servicios.
Usuario bloqueado	Cinco intentos fallidos en 15 minutos.	Esperar 15 minutos, reiniciar el servicio (borra los contadores) o recuperar la contraseña.

Nota. Elaboración propia con base en las decisiones documentadas.

16.7 Evolución recomendada

La documentación identifica como evolución natural, si el proyecto sale del ámbito académico: firma RS256, refresh tokens, Outbox para eliminar la ventana de pérdida de eventos, una base de datos física por servicio, mTLS entre servicios, observabilidad (trazado y métricas) y una pasarela de pagos. Cada una exige una ADR que responda a las cuatro preguntas de la guía de patrones (problema, requisito, complejidad, operabilidad).

16.8 Decisiones pendientes

- Ubicación final, formatos de imagen aceptados y tamaño máximo de los comprobantes de pago.
- Si la ejecución del servicio puede comenzar antes de la aprobación del pago.
- Criterio de desempate en la asignación automática de operarios (los documentos actuales son inconsistentes).
- Si se usarán permisos granulares o solo roles.
- Texto de la respuesta de cada endpoint y publicación de los seis contratos OpenAPI.

17. Glosario

Tabla 35
Glosario técnico

Término	Definición
ADR	Registro de decisión de arquitectura (Architecture Decision Record).
Agregado	Grupo de objetos de dominio que deben mantenerse consistentes juntos y tienen una raíz (Evans, 2004). El principal es Booking.
API Gateway	Punto único de entrada externo que enruta peticiones y valida el JWT en el borde.
Arquitectura hexagonal	Estilo de puertos y adaptadores en que el dominio no depende de la infraestructura (Cockburn, 2005).
Bahía	Puesto físico de lavado; su cantidad es la capacidad del negocio.
Bloqueo optimista	Control de concurrencia con <i>row_version</i> : una actualización sobre datos desactualizados no afecta ninguna fila.
Borrado lógico	Marca de eliminación mediante <i>deleted_at</i> en lugar de borrar la fila.
Contexto delimitado	Frontera semántica dentro de la cual un modelo de dominio es coherente.
Correlation id	Identificador que acompaña una petición a través del gateway, los servicios y los mensajes.
Cola de mensajes fallidos (DLQ)	Cola donde se conservan los mensajes que no pudieron procesarse tras los reintentos.
Evento de dominio	Hecho de negocio ocurrido, publicado para que otros servicios reaccionen.
Exchange topic	Intercambio de RabbitMQ que enruta mensajes por claves de enrutamiento.
Idempotencia	Propiedad de procesar un mismo evento más de una vez sin efectos duplicados.
JWT	JSON Web Token: token firmado que transporta la identidad y el rol (Jones et al., 2015).
Liquibase	Herramienta de versionado y migración del esquema de base de datos.
Libro de puntos	Tabla <i>payment.loyalty_transaction</i> , fuente de verdad del saldo de fidelidad.
Objeto de valor	Objeto definido por sus valores y no por una identidad (por ejemplo, Money o TimeSlot).
Outbox	Patrón que guarda los eventos en la misma transacción antes de publicarlos; no adoptado.
Puerto y adaptador	Interfaz que el dominio necesita y su implementación concreta en la infraestructura.
QR	Código bidimensional que se escanea para realizar el pago.
Referencia entre servicios	Columna con índice y sin llave foránea que apunta a datos de otro servicio.
SESSION_CONTEXT	Mecanismo de SQL Server para pasar el usuario actual a los disparadores de auditoría.

Término	Definición
Soft delete	Véase borrado lógico.

Nota. Elaboración propia con base en el glosario del proyecto y las referencias citadas.

18. Anexos

Anexo A. Lista de figuras pendientes de captura

Antes de la entrega, el equipo debe reemplazar cada espacio reservado por una imagen real y legible: los diagramas C4 y de flujo exportados desde sus fuentes, los diagramas entidad-relación exportados desde los archivos DBML y la captura de la documentación interactiva de la API. Las imágenes no deben contener secretos, cuentas bancarias ni datos personales reales.

Anexo B. Discrepancias detectadas en la documentación

La Tabla 36 reúne las contradicciones encontradas entre documentos del repositorio. Cada una debe resolverse antes de la entrega final.

Tabla 36***Discrepancias de la documentación y decisión adoptada en este manual***

Tema	Discrepancia	Criterio usado aquí
Recuento de tablas	Arquitectura indica 43, models.md indica 44 y las ADR-010 y ADR-011 añaden cinco tablas más (user_consent, person_address, service_reassignment, device_token y processed_event).	Se documentan las 44 de models.md y se citan las cinco adicionales; verificar contra los changelogs.
Estado de ADR-009	Figura como Propuesta, pero todos los documentos la usan como vigente.	Se trata como vigente.
Códigos de estado	ADR-010 define códigos en inglés para reservas; entities-and-rules usa español para reservas y ejecución.	Reservas en inglés; ejecución según la fuente, pendiente de confirmar.
Contrato de autenticación	El OpenAPI y el README del servicio de autenticación describen RS256, refresh token de 7 días, PostgreSQL, Redis y UUID; la ADR-006 fija HS256, sin refresh token, SQL Server e IDENTITY.	Prevalece la ADR-006.
Plantillas sin adaptar	El catálogo de servicios, el README del gateway, local-setup y el .env.example contienen PostgreSQL, Redis, Kafka, límite de peticiones y puertos de plantilla.	Se ignoran como plantillas.
Nombres de roles	ADMIN, OPERATOR y CLIENT en la política de seguridad; Administrator, Operator y Customer en las ADR; ADMIN, USER y VIEWER en el contrato del gateway.	ADMIN, OPERATOR y CLIENT.
Eventos y exchange	La plantilla de eventos usa auth.events y claves como user.registered; la arquitectura define carwash.events y security.user_registered.	Prevalece la arquitectura.
Outbox	La plantilla de eventos lo recomienda; la arquitectura indica que no está adoptado.	No adoptado.
Asignación automática	El criterio de desempate no está definido y los requisitos son inconsistentes.	Solo por disponibilidad; pendiente.
Lenguaje de los servicios	La documentación afirma Java 21 y Spring Boot en los seis servicios.	Confirmar contra el código entregado.
Esquema de loyalty_movement_type	En models.md la FK de loyalty_transaction apunta a customer.loyalty_movement_type, pero la tabla pertenece al esquema payment desde ADR-009.	Se trata como payment.loyalty_movement_type; corregir el texto de origen.
Ejecución de las migraciones	La estrategia define un único changelog maestro, pero no indica qué proceso lo ejecuta (paso de despliegue dedicado o arranque de un servicio).	Se documenta el changelog único; confirmar quién lo ejecuta.
Puertos	No hay puertos fijados para los seis servicios; solo el 8080 de la plantilla del gateway.	No se fijan puertos.

Nota. Elaboración propia a partir del análisis cruzado de la documentación del proyecto.

Anexo C. Lista de chequeo de entrega

Tabla 37***Lista de chequeo del manual técnico***

Criterio	Cumple
Portada completa (ficha, instructor y centro diligenciados) y versión	Sí / No
Tabla de contenido con páginas actualizadas	Sí / No
Objetivo y alcance explicados	Sí / No
El contenido coincide con el código de la versión final	Sí / No
Todas las figuras reemplazadas por imágenes reales	Sí / No
Estructura del proyecto verificada contra el código real	Sí / No
Discrepancias del Anexo B resueltas	Sí / No
Sin credenciales, claves ni datos sensibles	Sí / No
Ortografía y presentación revisadas	Sí / No
Exportado en PDF con el nombre Manual_tecnico_NombreProyecto_Ficha.pdf	Sí / No

Nota. Adaptada de la lista de chequeo de la guía ADSO (SENA, 2026).

Referencias

- Cockburn, A. (2005). *Hexagonal architecture*. <https://alistair.cockburn.us/hexagonal-architecture/>
- Congreso de la República de Colombia. (2012). *Ley Estatutaria 1581 de 2012, por la cual se dictan disposiciones generales para la protección de datos personales*.
- Equipo desarrollador. (2026). *Documentación del Sistema de Gestión de Lavado Vehicular (Vehicle Washing)* [Repositorio de documentación del proyecto]. Programa Tecnólogo en Análisis y Desarrollo de Software, SENA.
- Evans, E. (2004). *Domain-driven design: Tackling complexity in the heart of software*. Addison-Wesley.
- Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Tesis doctoral, University of California, Irvine].
- Hohpe, G., y Woolf, B. (2003). *Enterprise integration patterns: Designing, building, and deploying messaging solutions*. Addison-Wesley.
- Jones, M., Bradley, J., y Sakimura, N. (2015). *JSON Web Token (JWT) (RFC 7519)*. Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc7519>
- Newman, S. (2021). *Building microservices: Designing fine-grained systems* (2.^a ed.). O'Reilly Media.
- OWASP Foundation. (2021). *OWASP Top 10:2021*. <https://owasp.org/Top10/>
- Provos, N., y Mazières, D. (1999). A future-adaptable password scheme. *Proceedings of the USENIX Annual Technical Conference, FREENIX Track*, 81-91.
- Servicio Nacional de Aprendizaje [SENA]. (2026). *Guía para elaborar manuales de software: Manual de usuario, manual técnico, manual de instalación y manual de administrador*. Tecnólogo en Análisis y Desarrollo de Software.